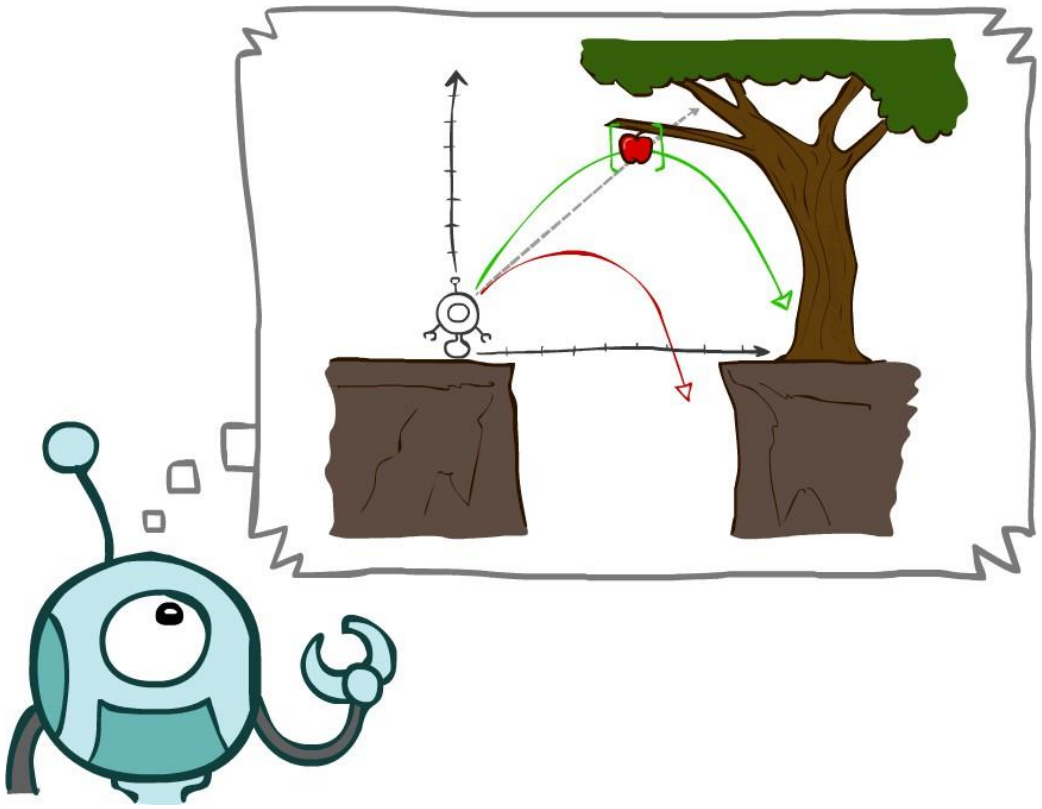
CSCI218: Foundations of Artificial Intelligence

UNIVERSITY
OF WOLLONGONG
AUSTRALIA

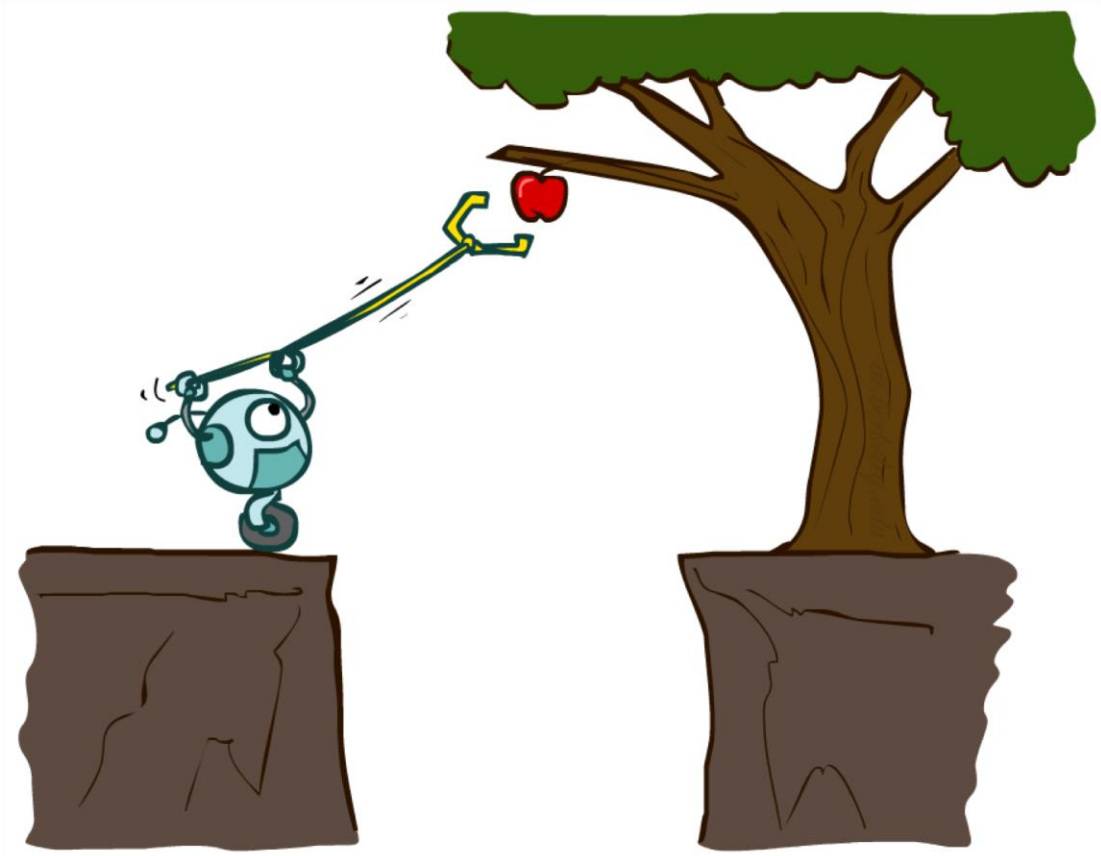
Solving Problems by Searching

- Problem-solving agent
- Search Problems
- Uninformed Search Methods
- Informed Search Methods



Problem-solving Agents

- Problem-solving agents decide based on evaluating action sequences
- Search algorithms typically assume
  - Known, deterministic transition model
  - Discrete states and actions
  - Fully observable
  - Atomic representation
- Usually have a definite goal
- Optimal: Achieve goal at the least cost



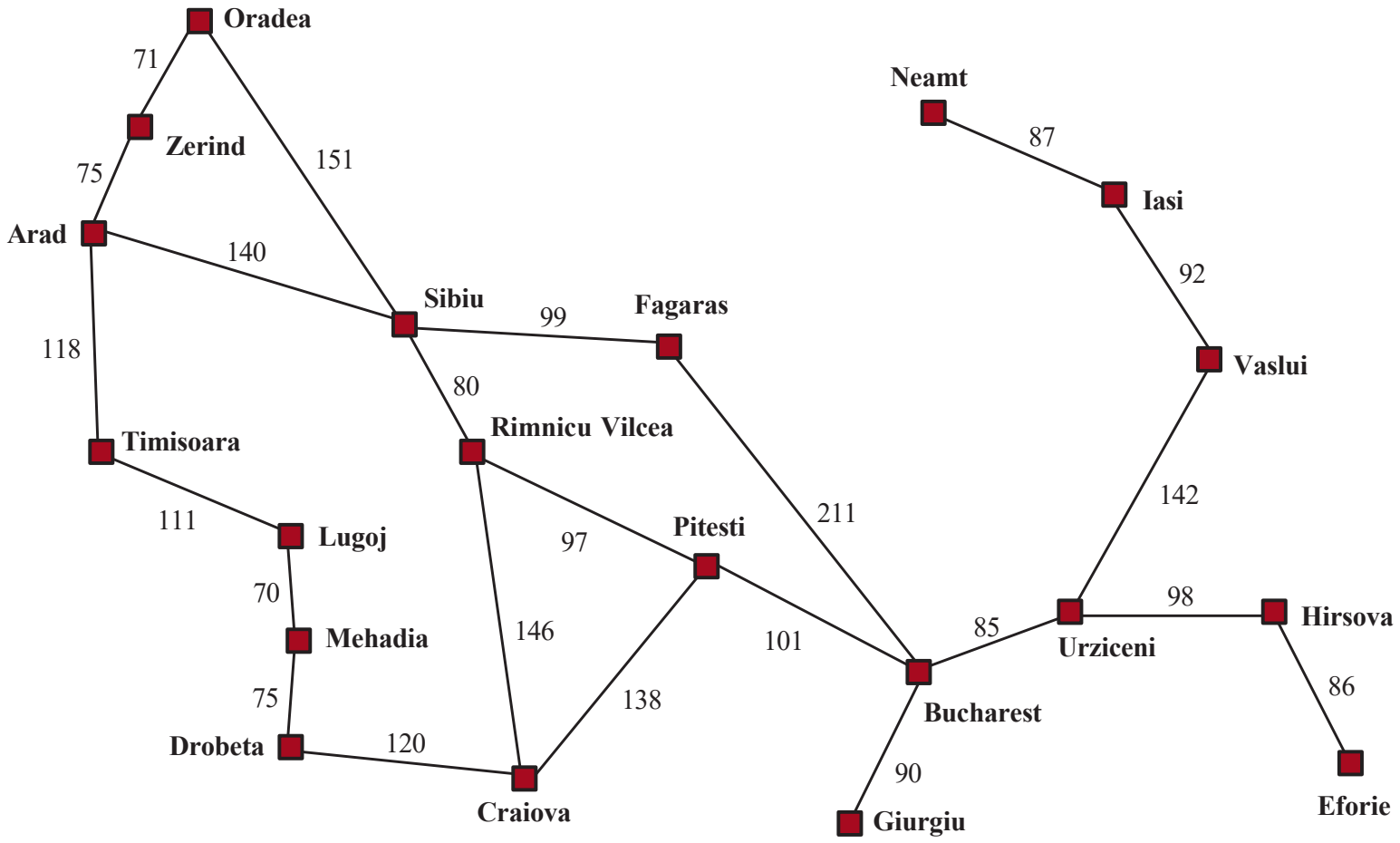
Search Problems

Search Problems

- A search problem consists of:
  - A state space $\mathcal{S}$
  - An initial state s_0
  - Actions $\mathcal{A}(s)$ in each state
  - Transition model $Result(s,a)$
  - A goal test $G(s)$
    - Test if s is the goal state
  - Action cost $c(s,a,s')$
    - E.+1 per step; -10 food; -500 win; +500 die; -200 eat ghost
- A solution is an action sequence that reaches a goal state
- An optimal solution has the least cost among all solutions

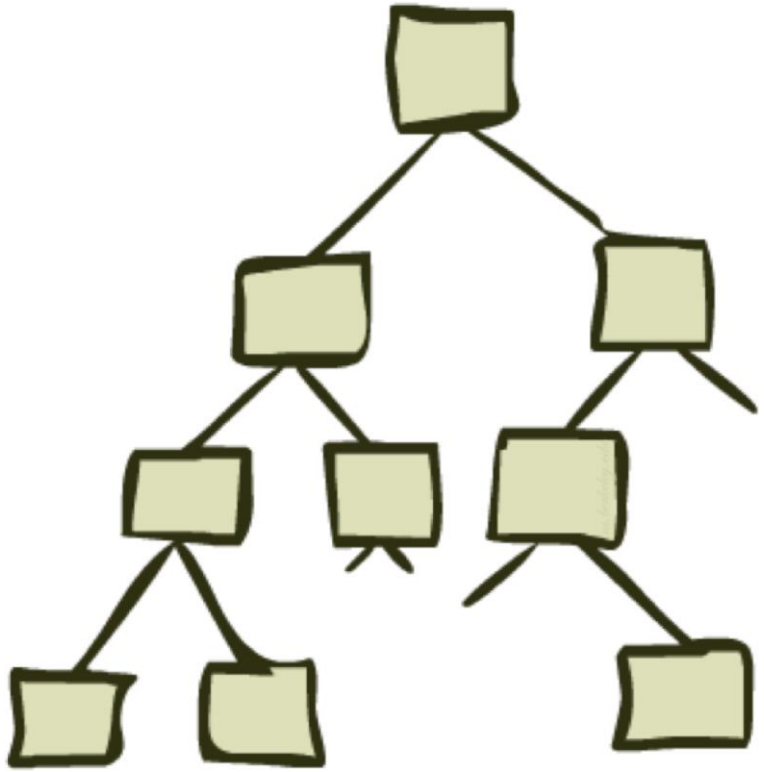
Example: Traveling in Romania

Example: Traveling in Romania



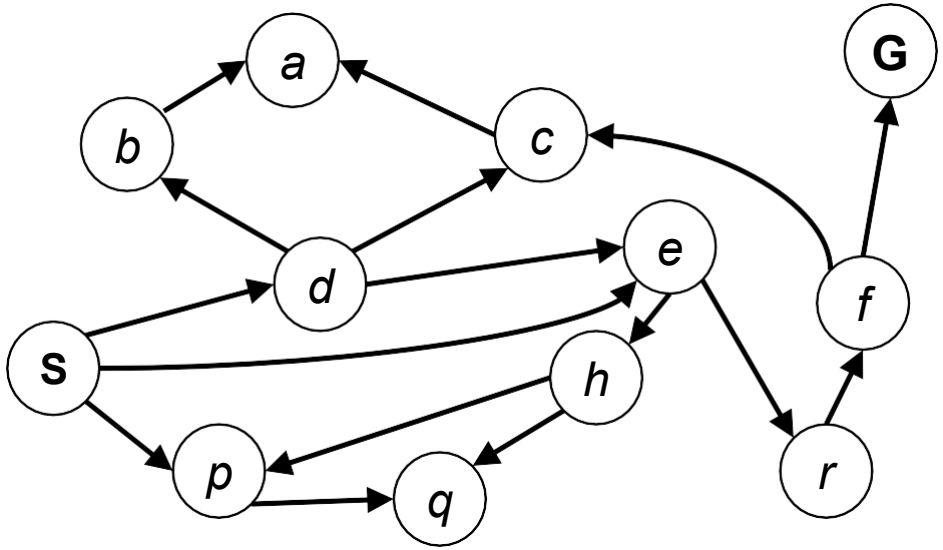
- State space:
  - Cities
- Initial state:
  - Arad
- Actions:
  - Go to adjacent city
- Transition model:
  - Reach adjacent city
- Goal test:
  - s = Bucharest?
- Action cost:
  - Road distance from s to s'
- Solution?

State Space Graphs and Search Trees



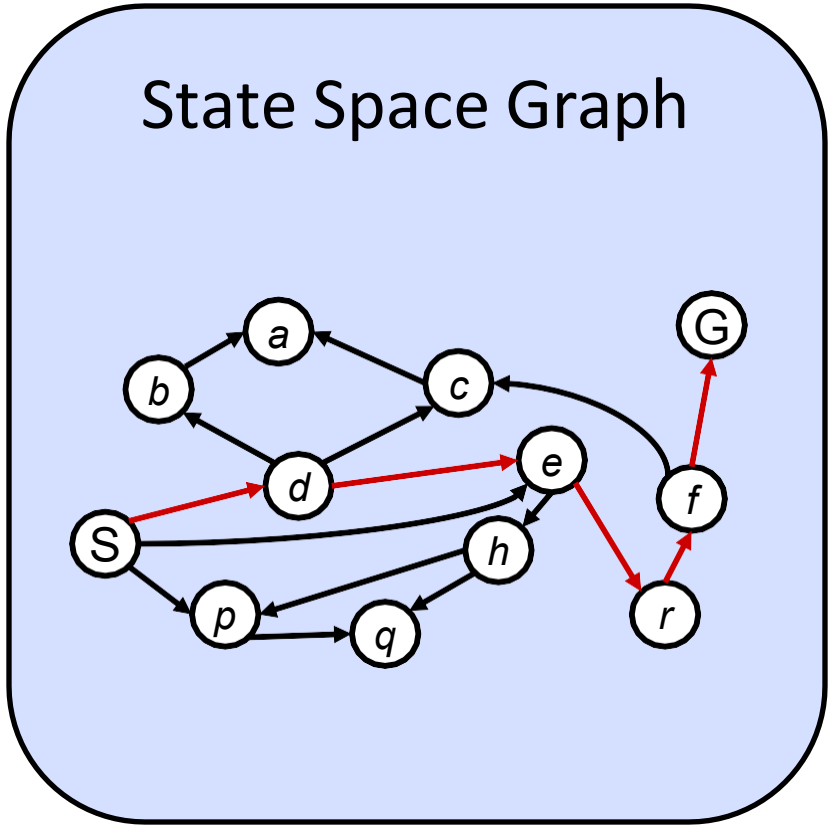
State Space Graphs

- State space graph: A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



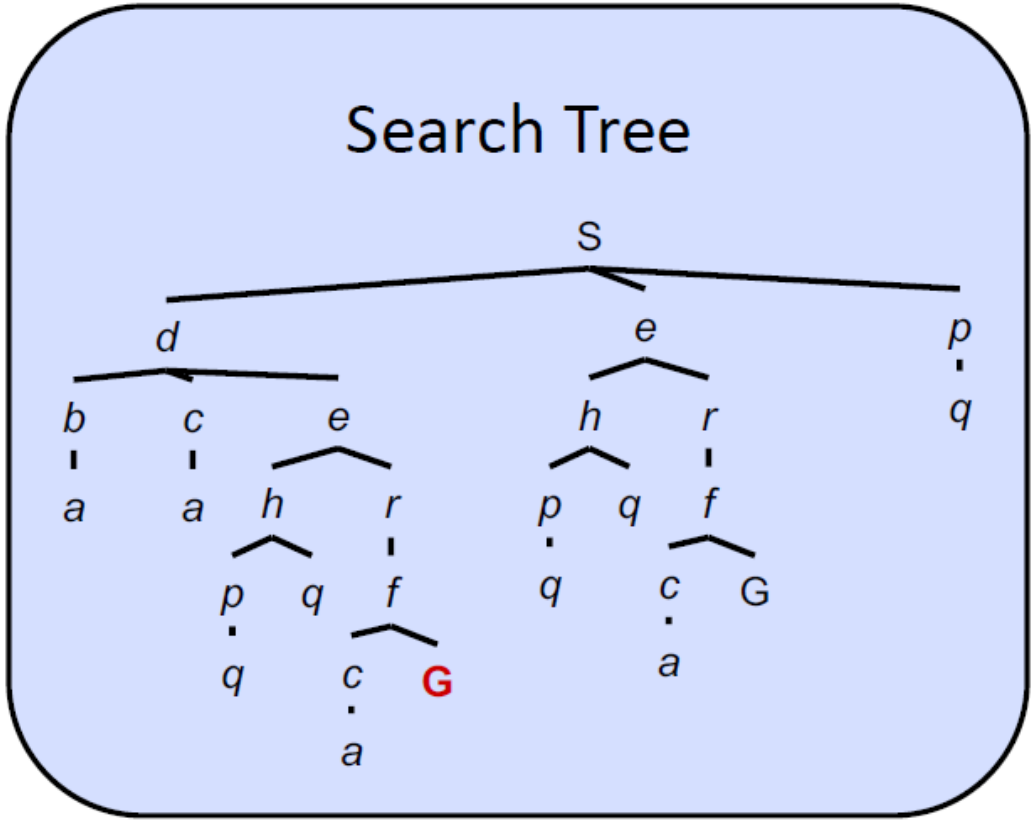
Tiny state space graph for a tiny search problem

State Space Graphs vs. Search Trees



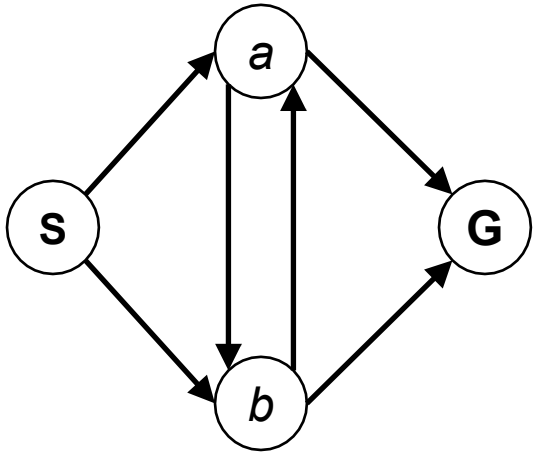
Each NODE in the search tree corresponds to an entire PATH in the state space graph.

We construct the tree on demand – and we construct as little as possible.



Quiz: State Space Graphs vs. Search Trees

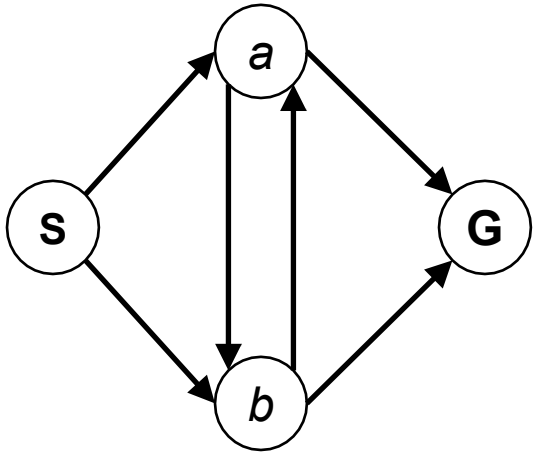
Consider this 4-state graph:



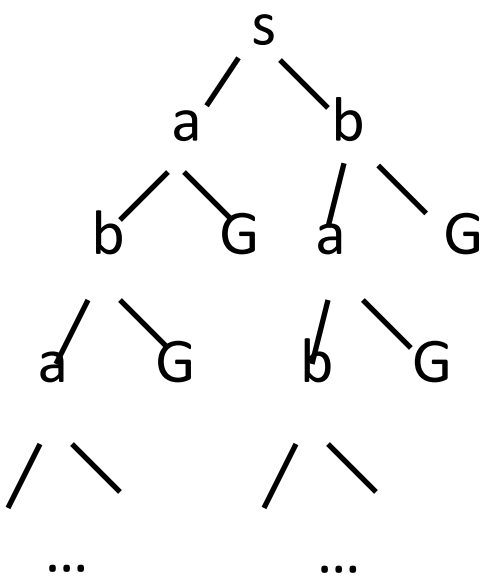
How big is its search tree (from S)?

Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:

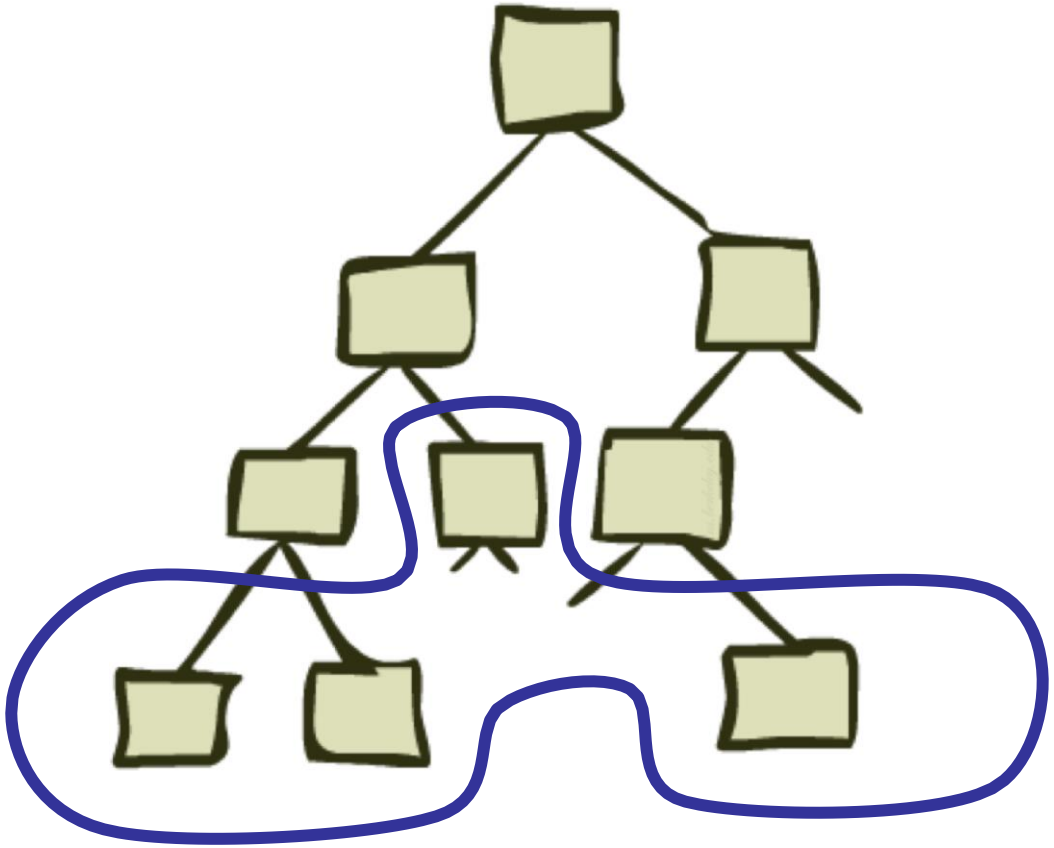


How big is its search tree (from S)?

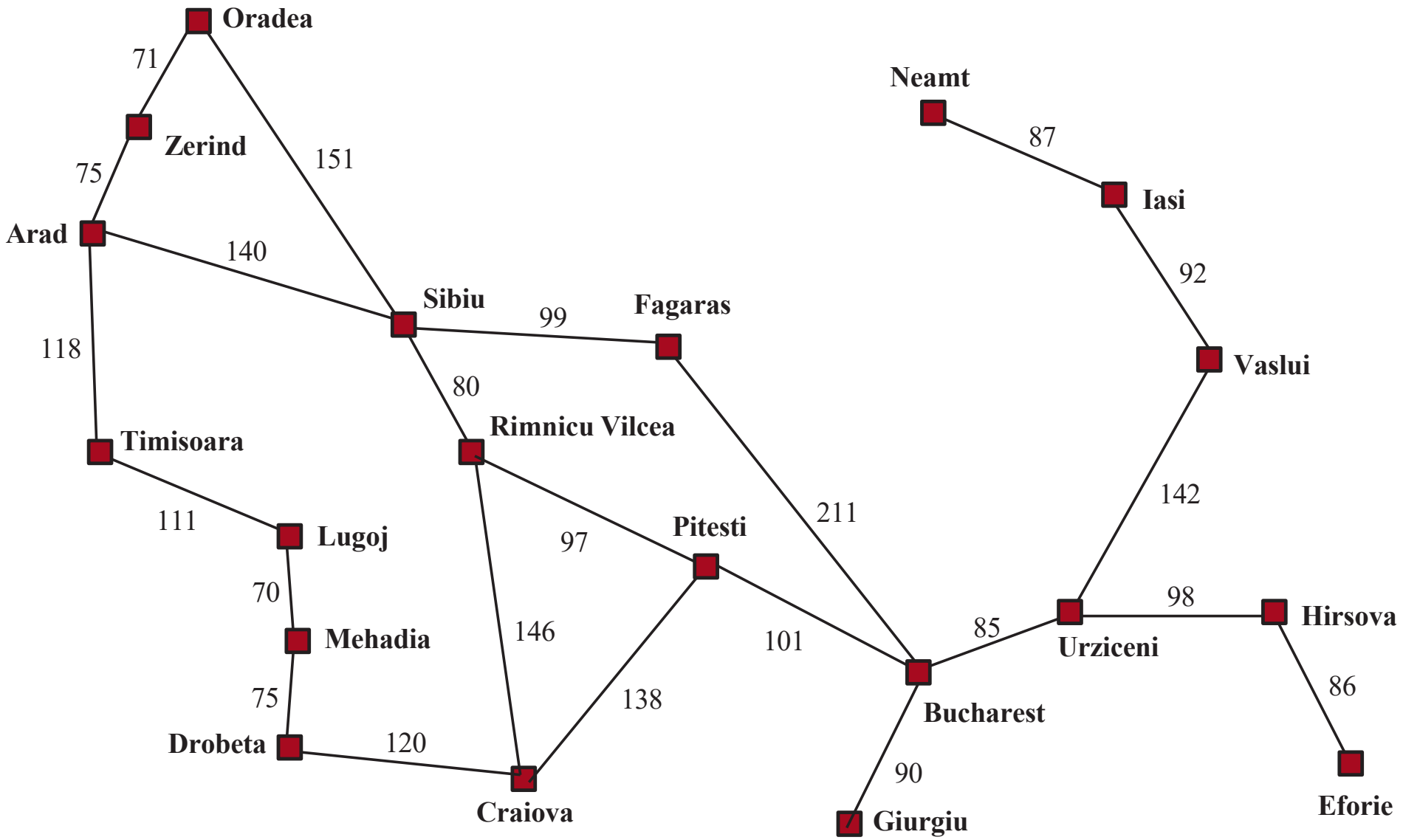


Important: Those who don’t know history are doomed to repeat it!

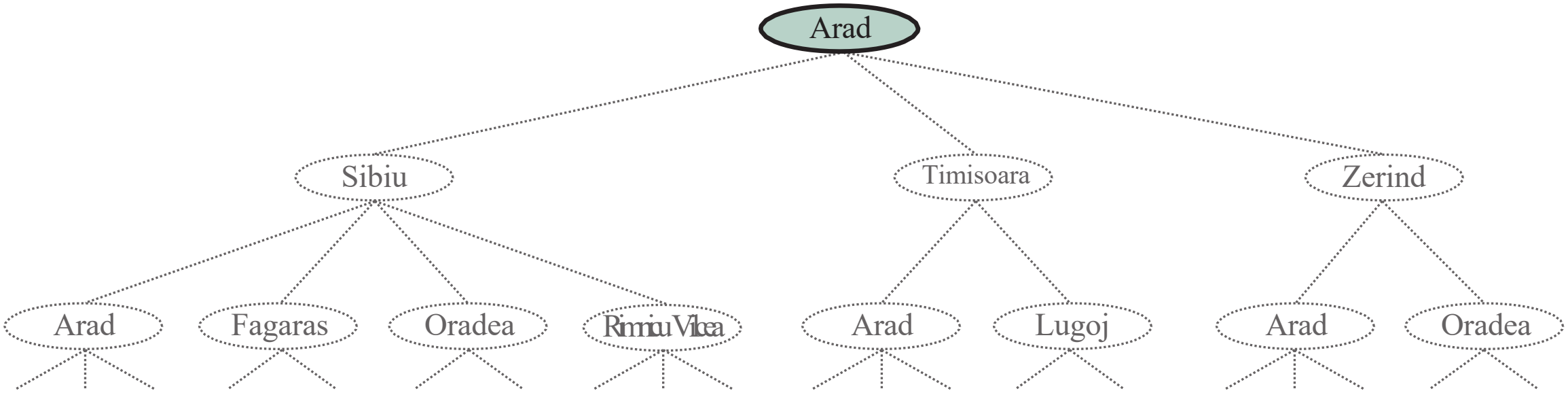
Tree Search



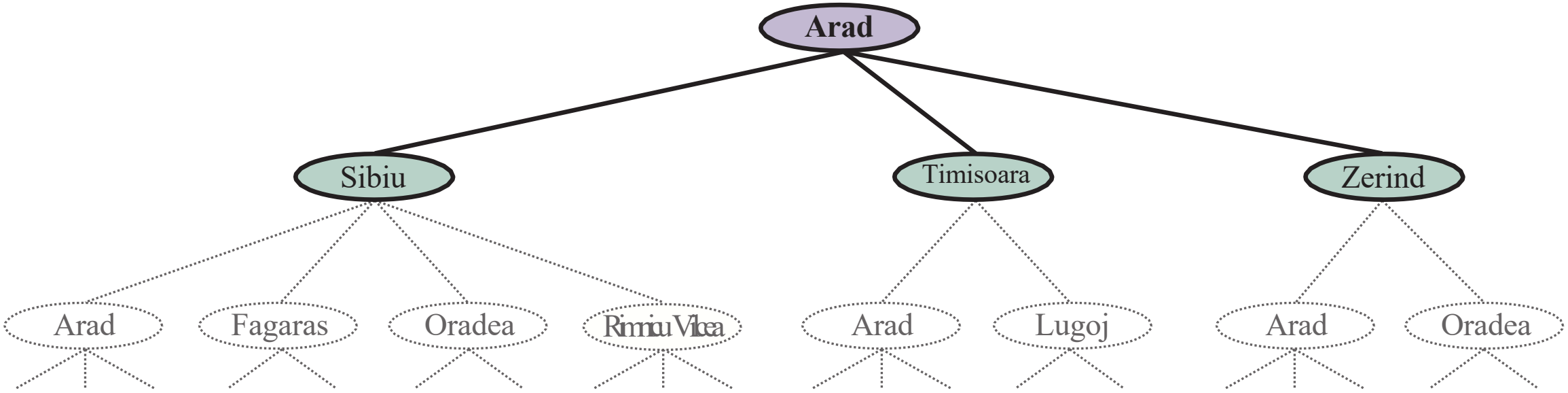
Search Example: Romania



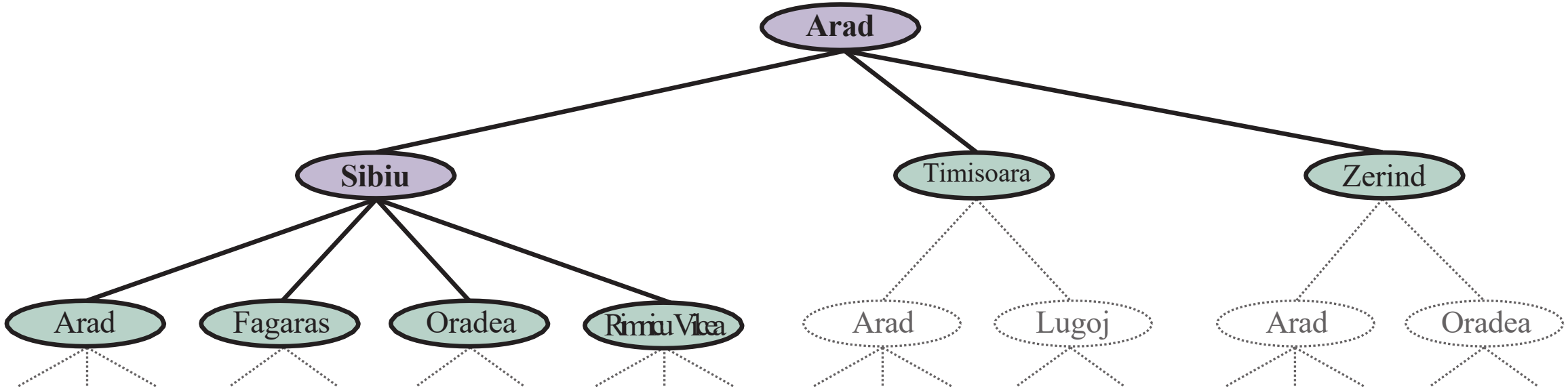
Creating the search tree



Creating the search tree



Creating the search tree

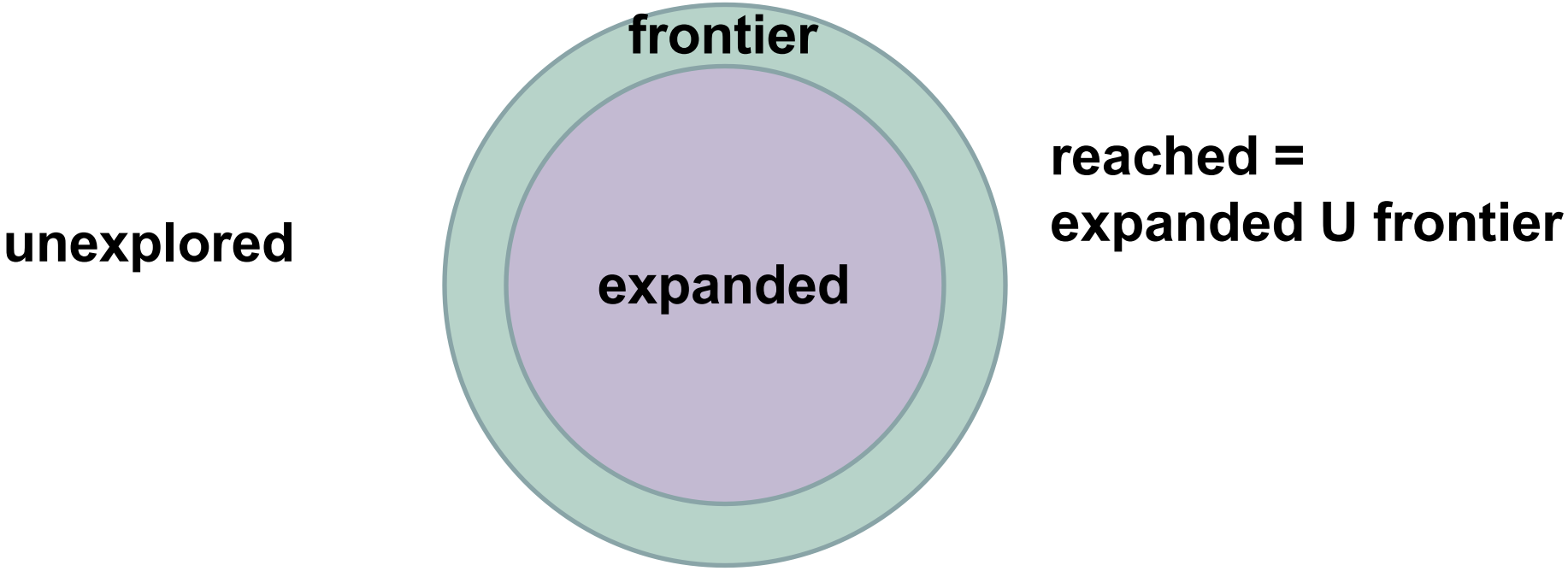


General Tree Search

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```

- Main variations:
  - Which leaf node to expand next
  - Whether to check for repeated states
  - Data structures for frontier, expanded nodes

Systematic search



1. Frontier separates expanded from unexplored region of state-space graph
2. Expanding a frontier node:
   a. Moves a node from frontier into expanded
   b. Adds nodes from unexplored into frontier, maintaining property 1

Search Strategies

- **A strategy is defined by picking the order of node expansion**

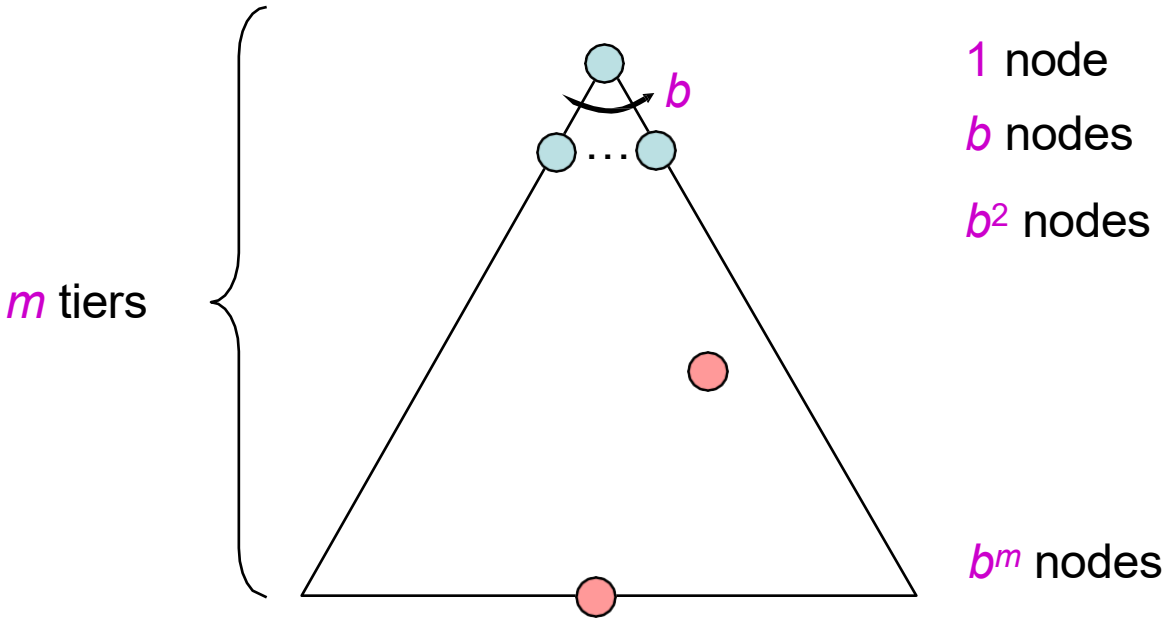
- **Strategies are evaluated along the following dimensions:**
  - completeness—does it always find a solution if one exists?
  - time complexity—number of nodes generated/expanded
  - space complexity—maximum number of nodes in memory
  - optimality—does it always find a least-cost solution?

- **Time and space complexity are measured in terms of**
  - b—maximum branching factor of the search tree
  - s—depth of the least-cost solution
  - m—maximum depth of the state space (may be ∞)

Search Algorithm Properties

- **Complete**: Guaranteed to find a solution if one exists?
- **Optimal**: Guaranteed to find the least cost path?
- Time **complexity**?
- Space **complexity**?

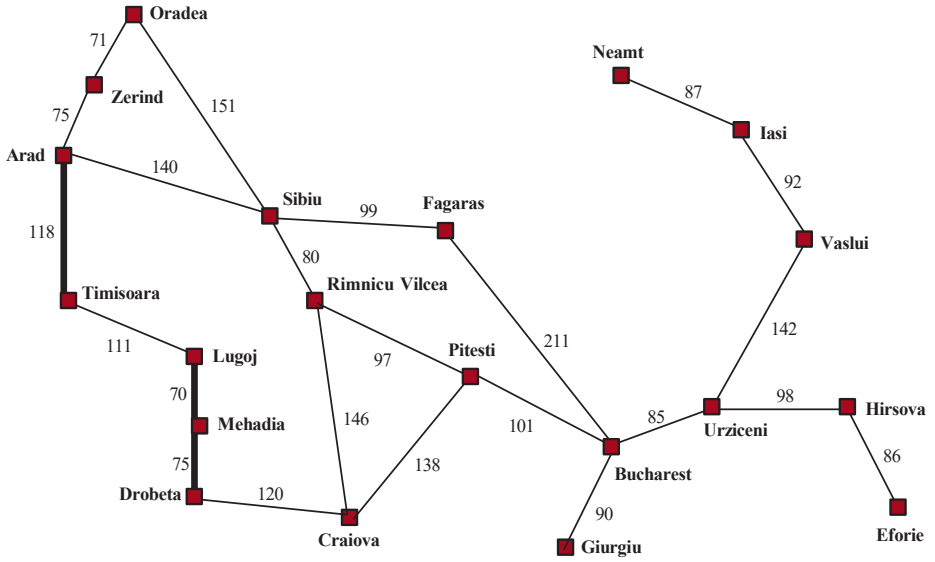
- Cartoon of search tree:
  - b is the branching factor
  - m is the maximum depth
  - solutions at various depths



- Number of nodes in entire tree?
  - $1 + b + b^2 + \ldots. b^m = O(b^m)$

Solving Problems by Searching

- Problem-solving agent
- Search Problems
- **Uninformed Search Methods**
  - No clue about how close a state is to the goal
  - E.g., no knowledge of Romanian geography
- Informed Search Methods
  - Knows the location of each city and which one is closer the Bucharest



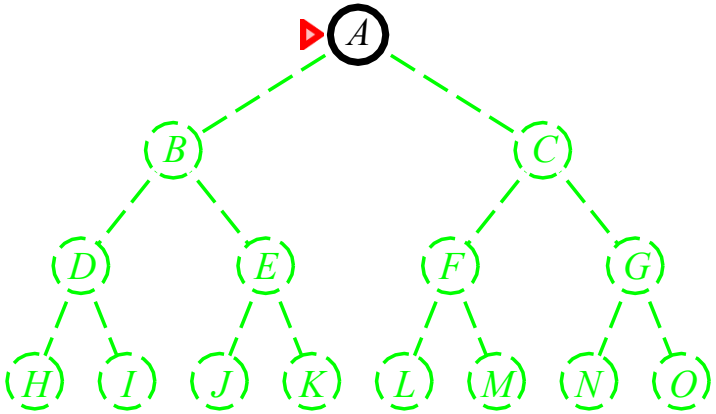
Depth-First Search

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



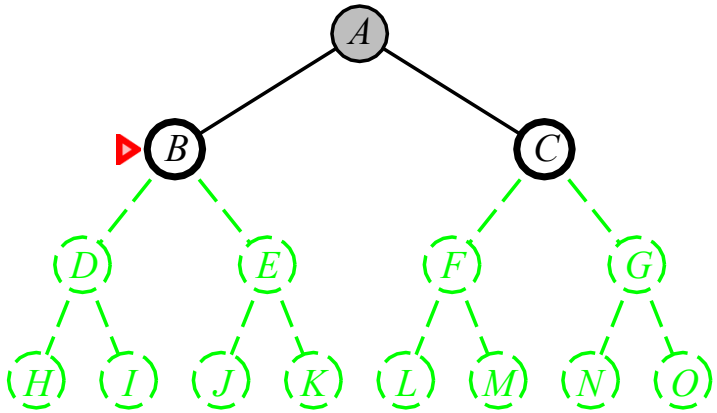
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



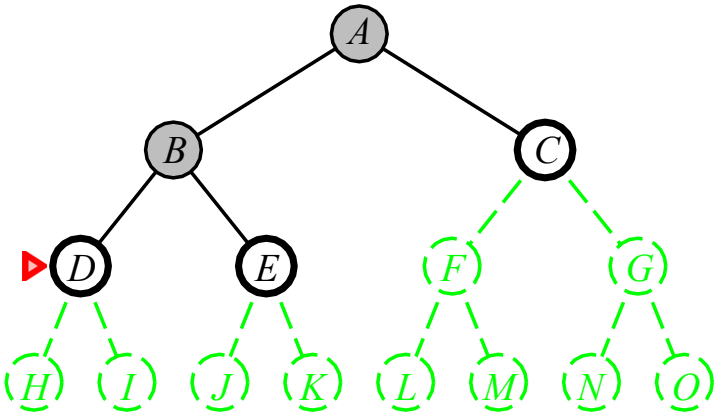
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



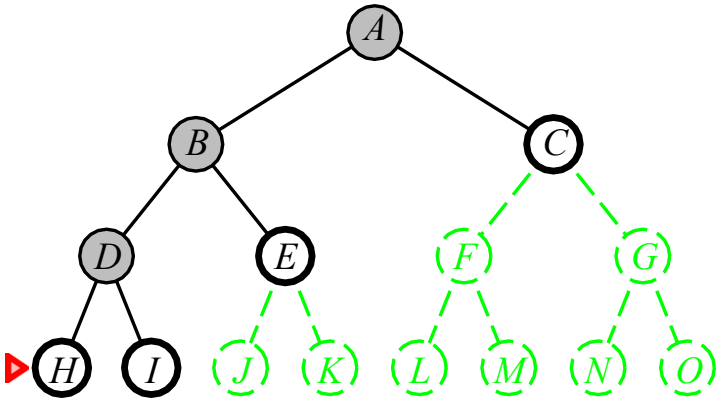
Pearson
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



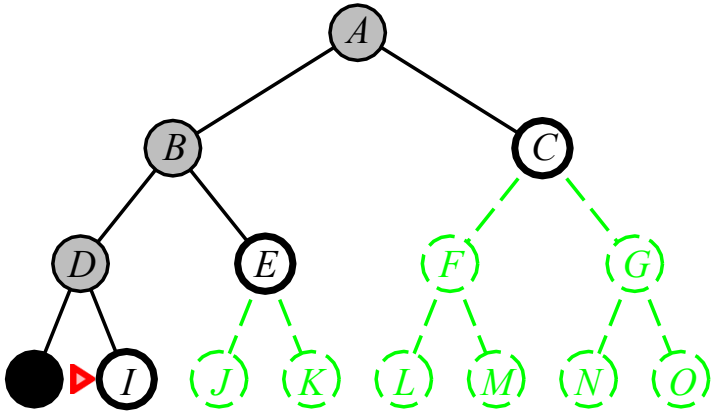
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



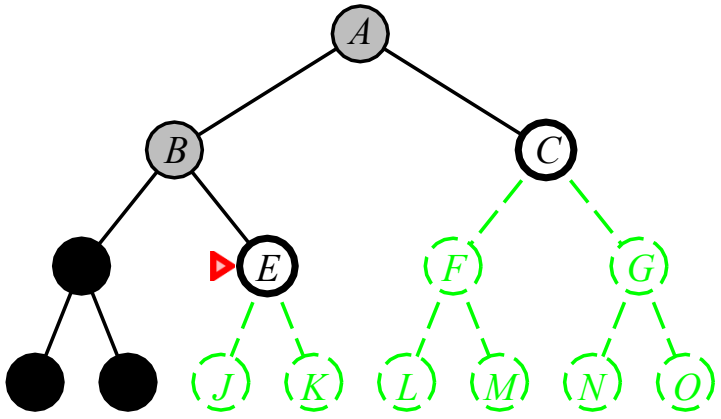
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



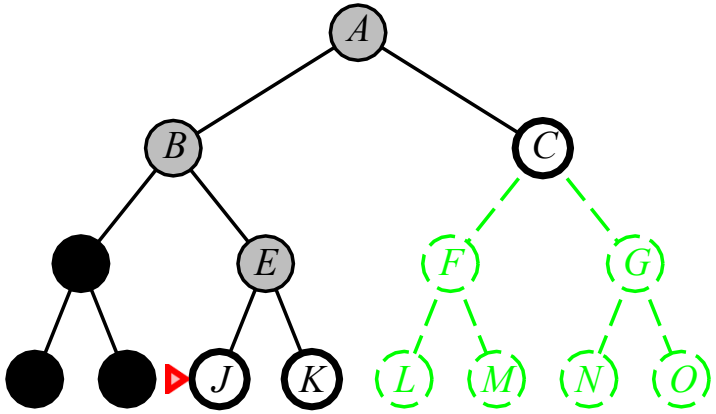
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



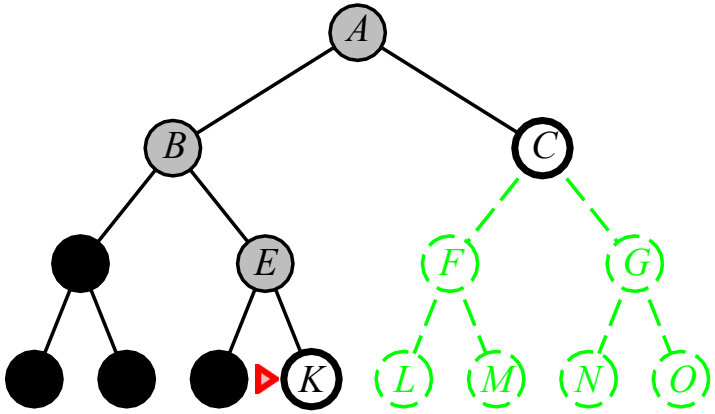
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



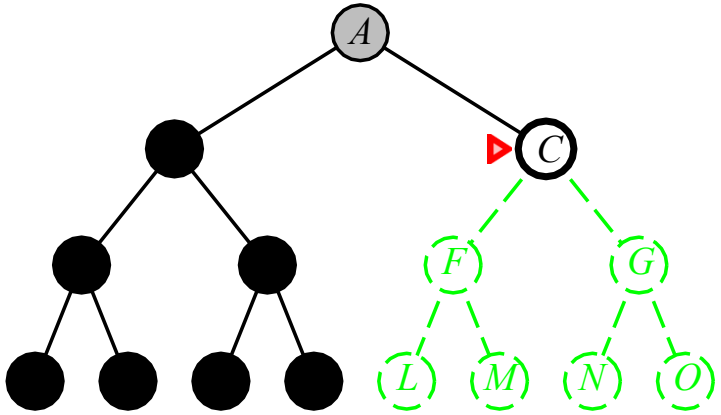
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



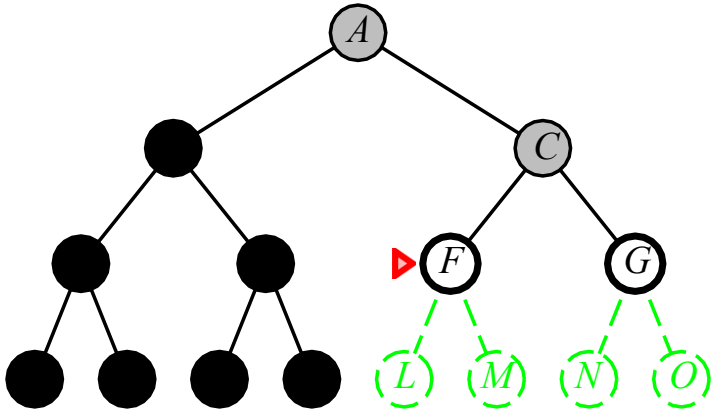
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



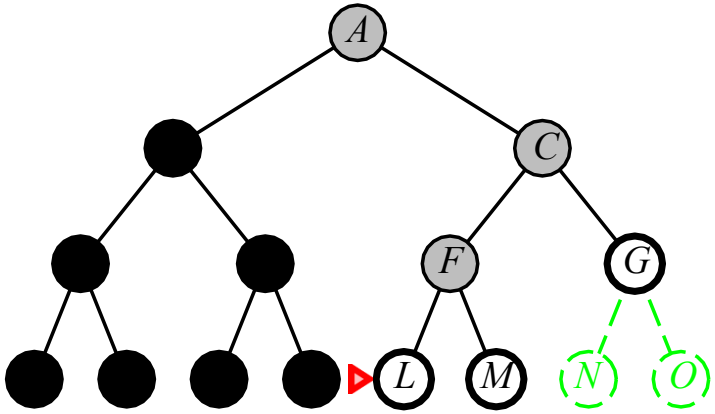
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front



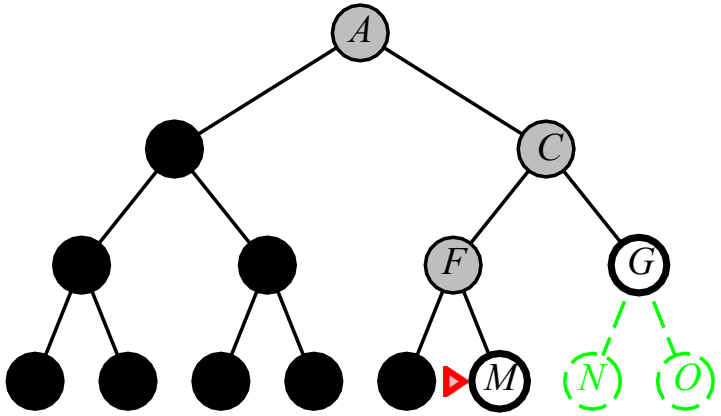
© 2022 Pearson Education Ltd.

Depth-First Search

Expand deepest unexpanded node

Implementation:

LIFO queue, i.e., put successors at front

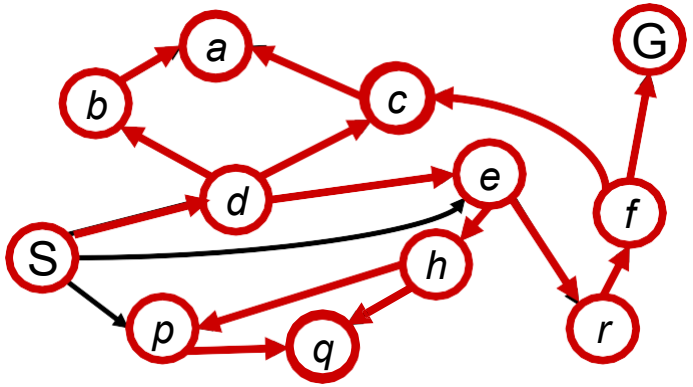


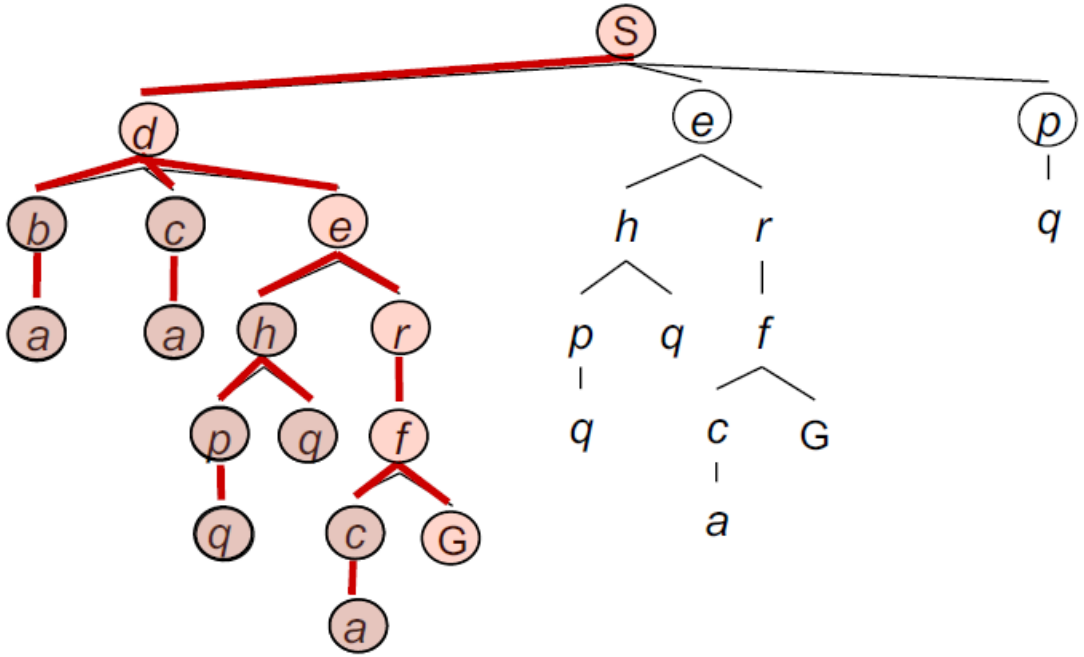
© 2022 Pearson Education Ltd.

Depth-First Search

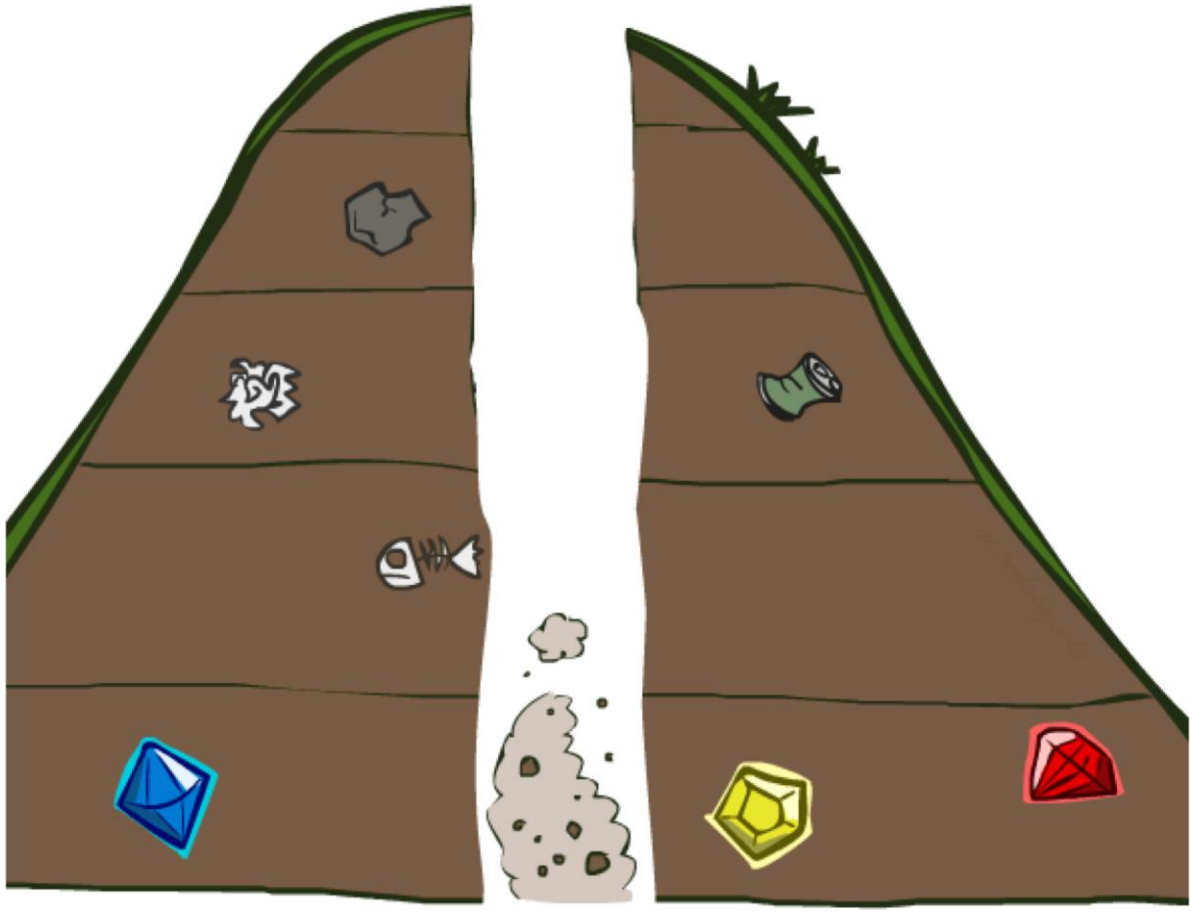
*Strategy: expand a **deepest** node first*

Implementation: Frontier is a LIFO (last in, first out) stack



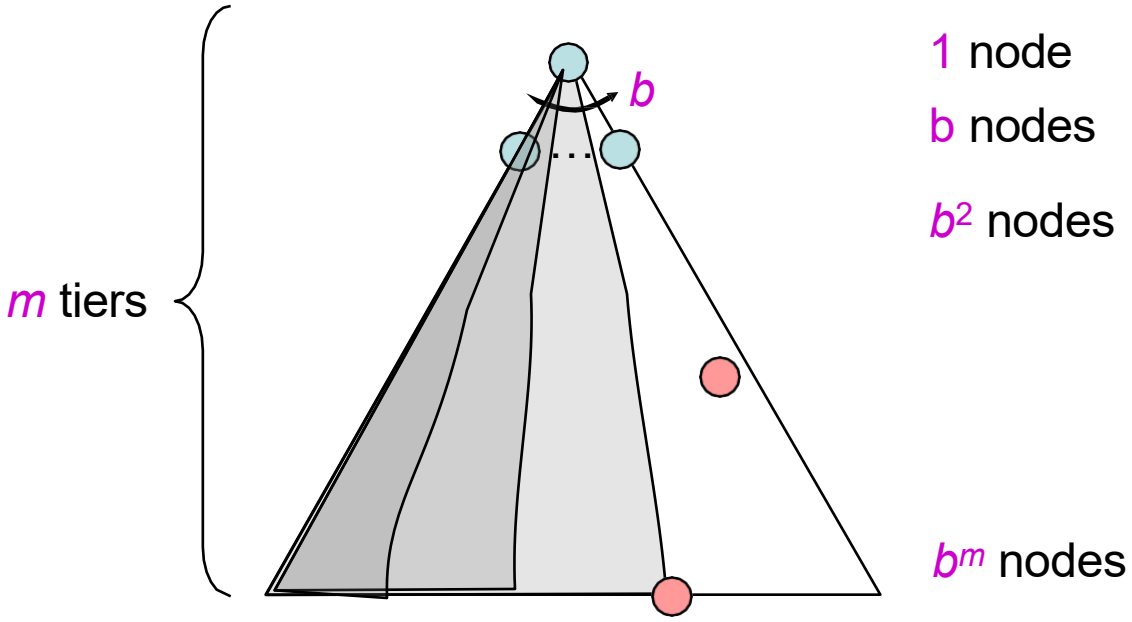


Search Algorithm Properties

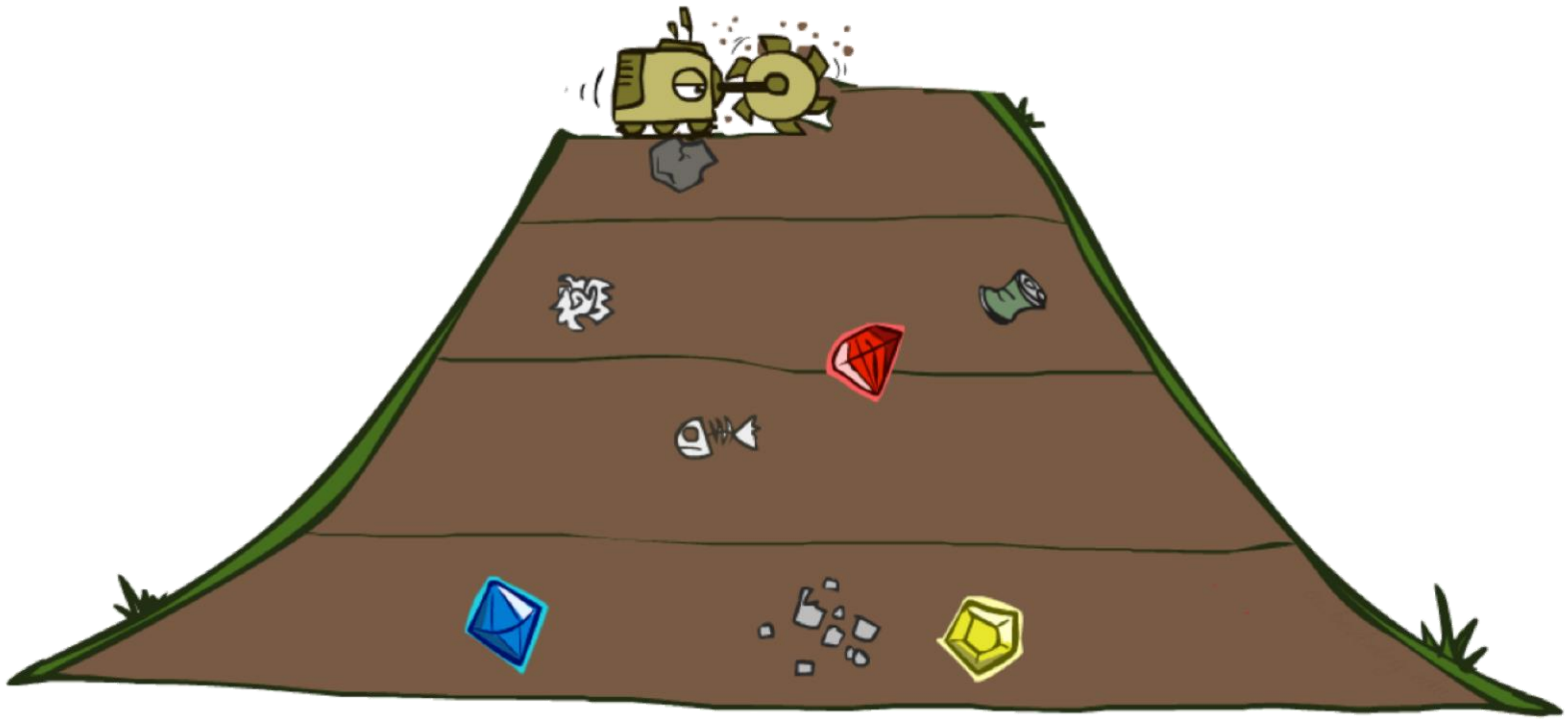


Depth-First Search (DFS) Properties

- What nodes does DFS expand?
  - Some left prefix of the tree down to depth m.
  - Could process the whole tree!
  - If m is finite, takes time $O(b^m)$
- How much space does the frontier take?
  - Only has siblings on path to root, so $O(bm)$
- Is it complete? (Depends…)
  - m could be infinite
  - preventing cycles may help (more later)
- Is it optimal?
  - No, it finds the "leftmost" solution, regardless of depth or cost



Breadth-First Search

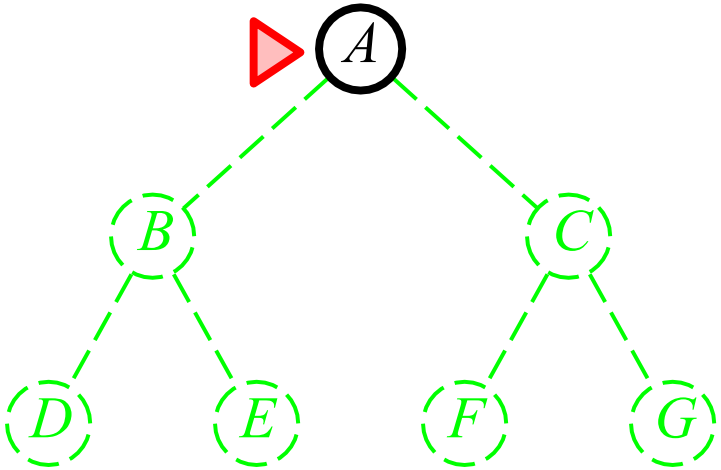


Breadth-First-Search

Expand shallowest unexpanded node

Implementation:

a FIFO queue, i.e., new successors go at end



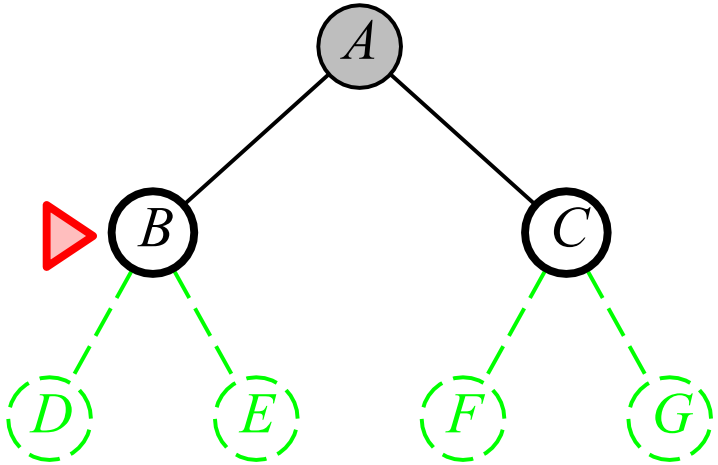
© 2022 Pearson Education Ltd.

Breadth-First-Search

Expand shallowest unexpanded node

Implementation:

a FIFO queue, i.e., new successors go at end



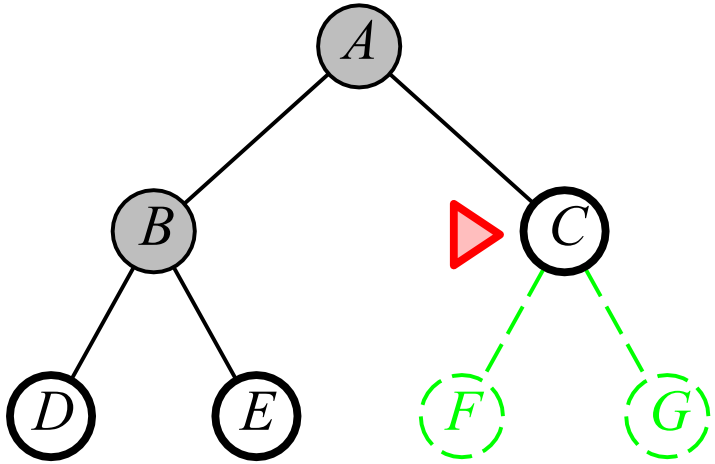
© 2022 Pearson Education Ltd.

Breadth-First-Search

Expand shallowest unexpanded node

Implementation:

a FIFO queue, i.e., new successors go at end



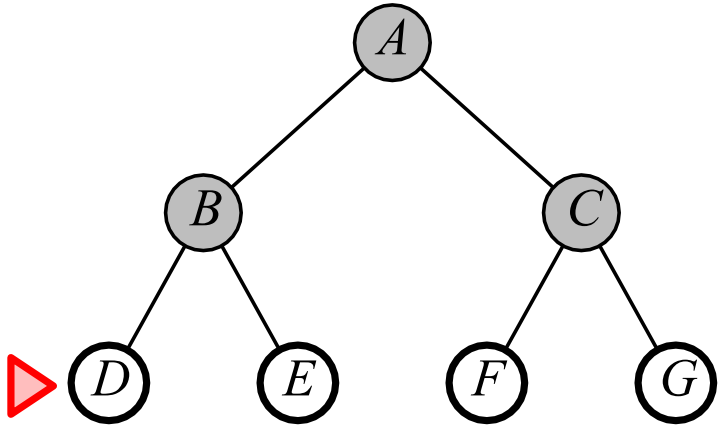
© 2022 Pearson Education Ltd.

Breadth-First-Search

Expand shallowest unexpanded node

Implementation:

a FIFO queue, i.e., new successors go at end

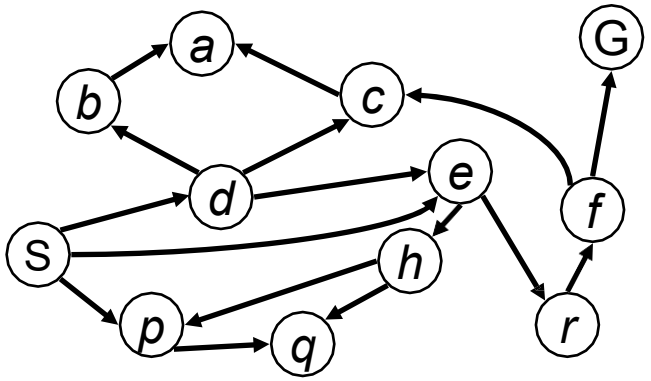


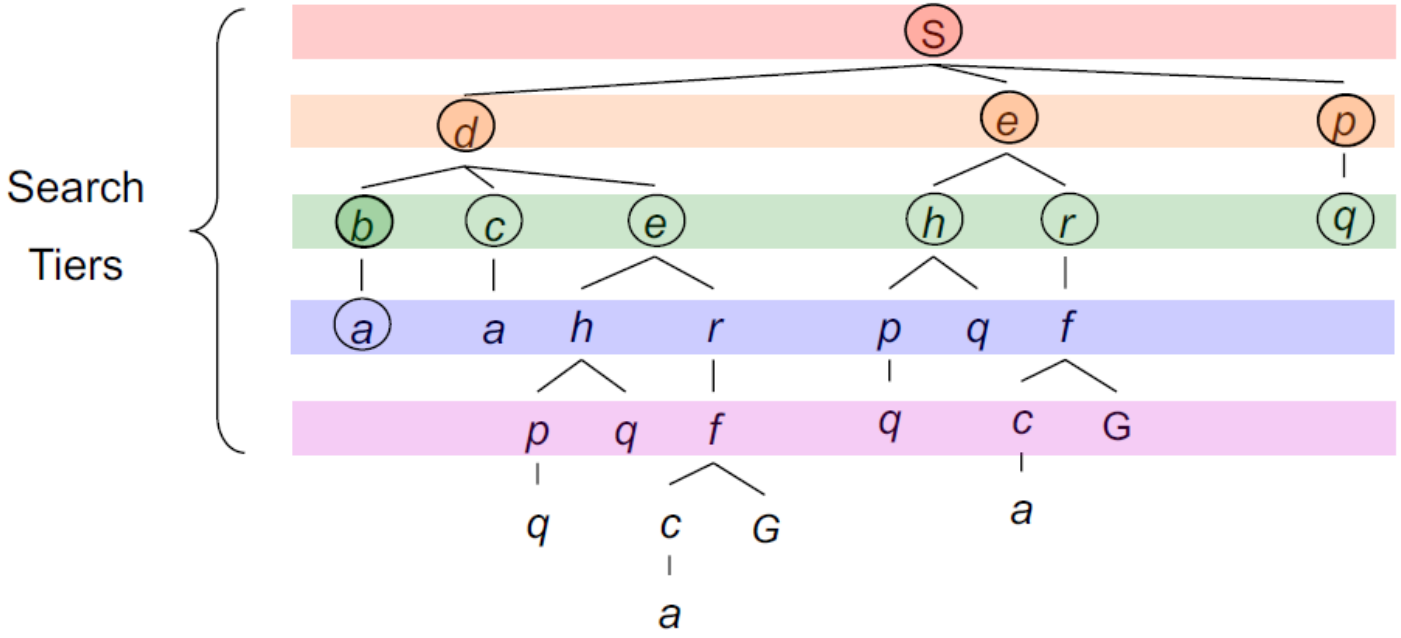
© 2022 Pearson Education Ltd.

Breadth-First Search

Strategy: expand a ***shallowest*** *node first*

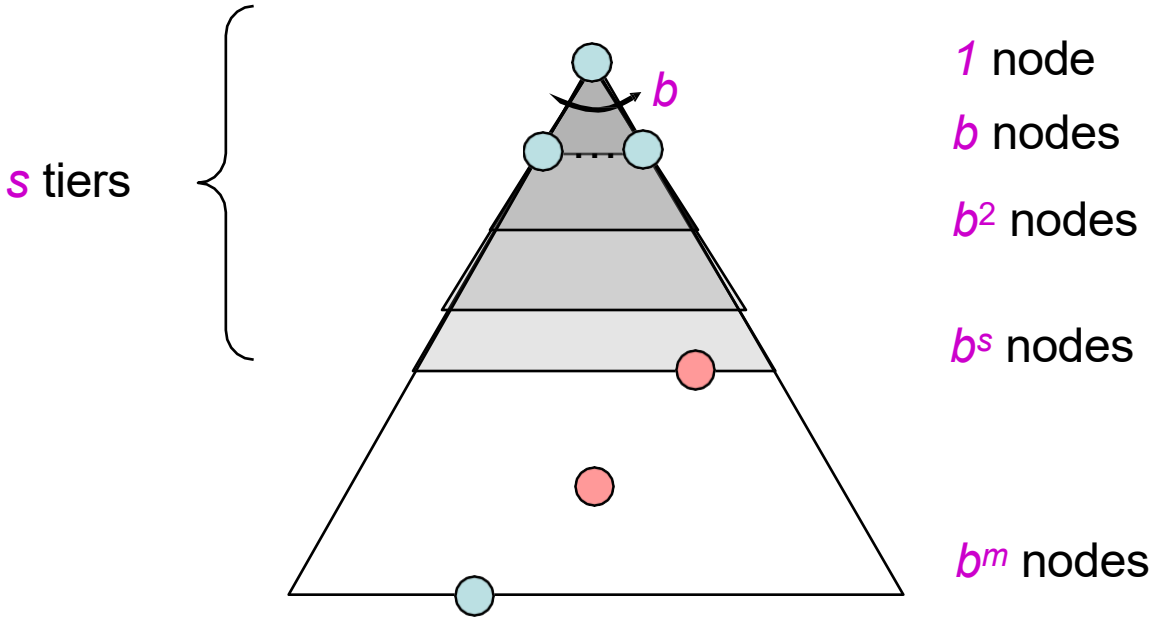
Implementation:
Frontier is a FIFO (First in, First out) queue



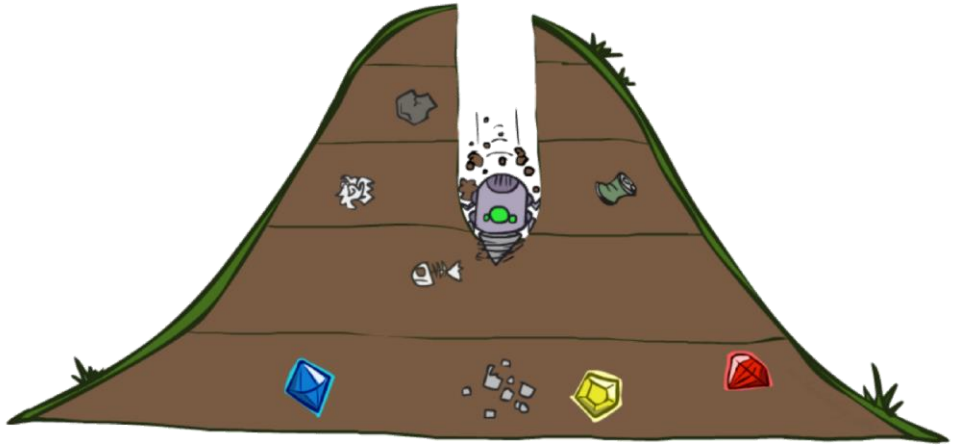


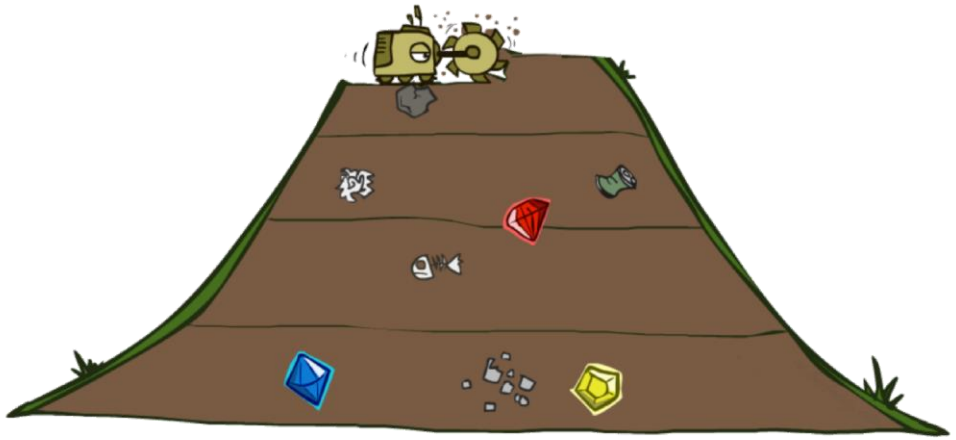
Breadth-First Search (BFS) Properties

- What nodes does BFS expand?
  - Processes all nodes above shallowest solution
  - Let depth of shallowest solution be s
  - Search takes time $O(b^s)$
- How much space does the frontier take?
  - Has roughly the last tier, so $O(b^s)$
- Is it complete?
  - s must be finite if a solution exists, so yes!
  - Complete even on infinite state spaces
- Is it optimal?
  - If all actions have the same cost (e.g., 1)



Quiz: DFS vs BFS





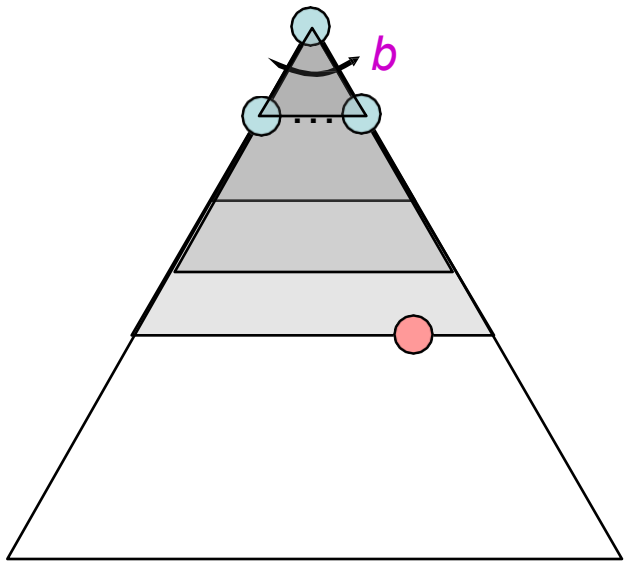
Quiz: DFS vs BFS

- When will BFS outperform DFS?

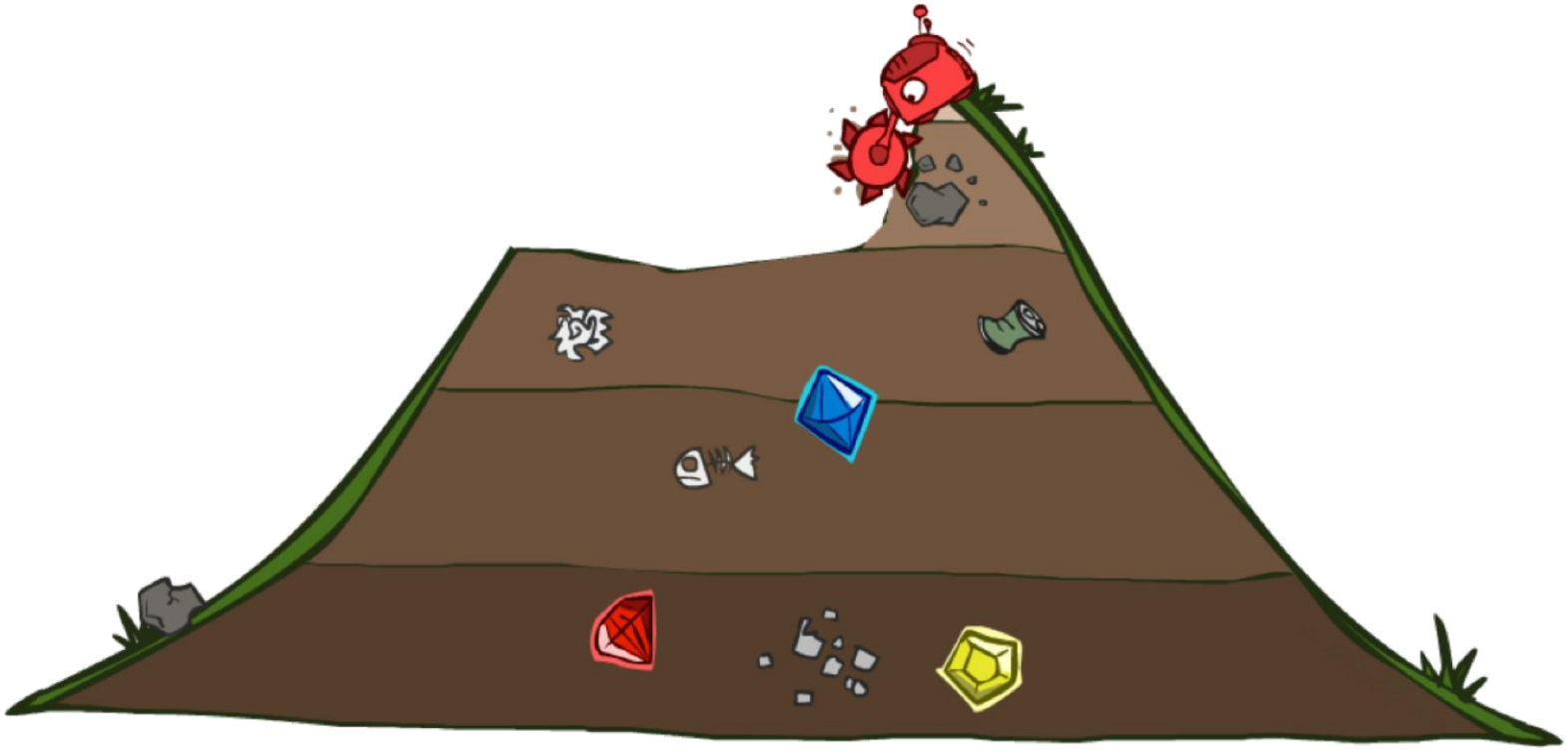
- When will DFS outperform BFS?

Iterative Deepening (IDF)

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
  - Run a DFS with depth limit 1. If no solution…
  - Run a DFS with depth limit 2. If no solution…
  - Run a DFS with depth limit 3. …..
- Isn't that wastefully redundant?
  - Generally most work happens in the lowest level searched, so not so bad!



Uniform Cost Search



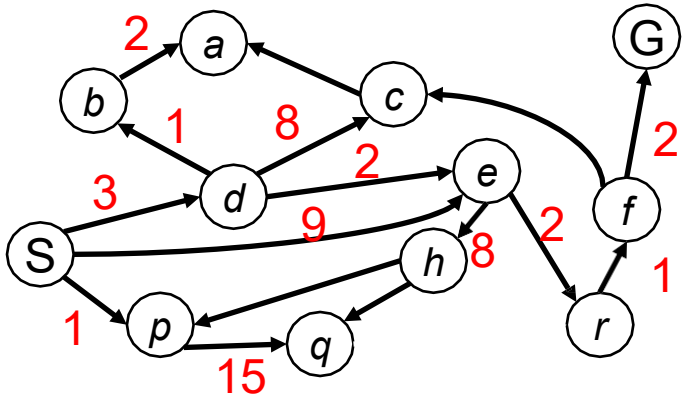
Uniform Cost Search

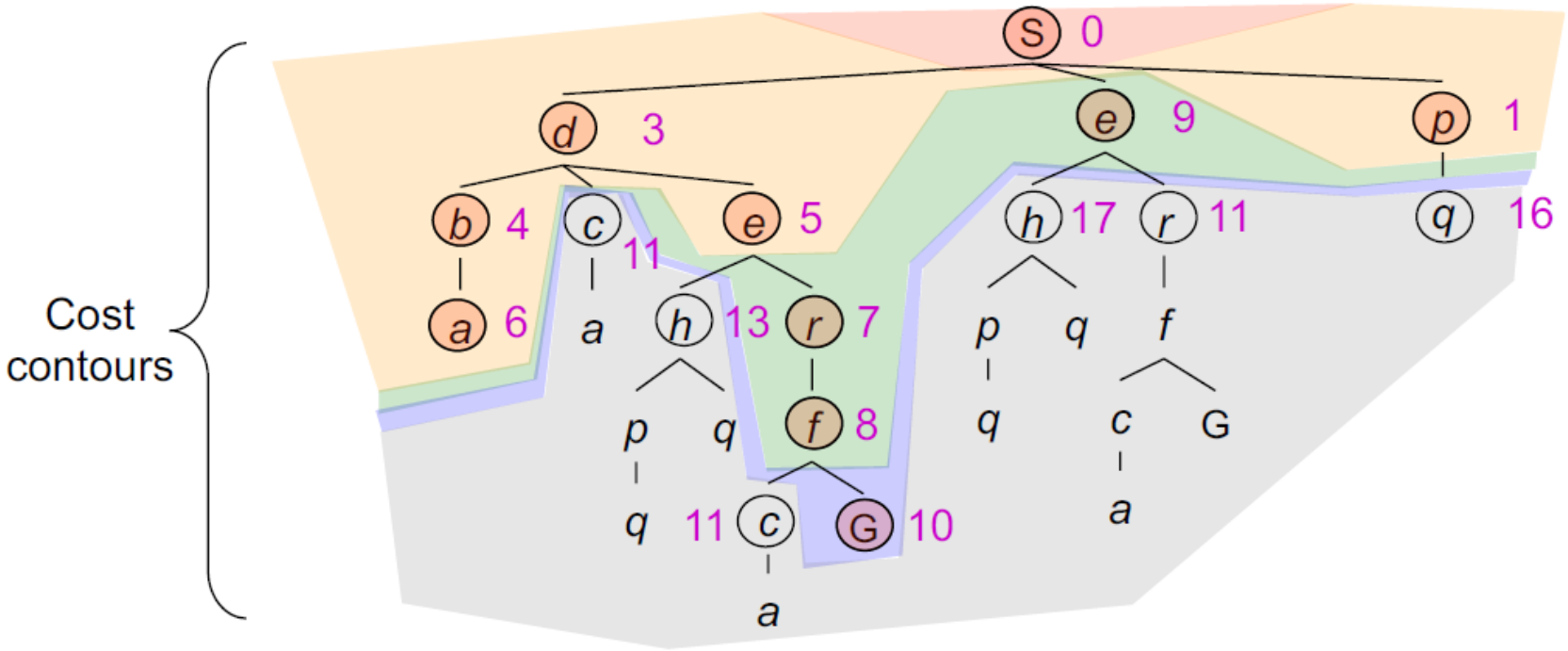
When actions have different costs

g(n) = cost from root to n

Strategy: ***expand lowest g(n)***

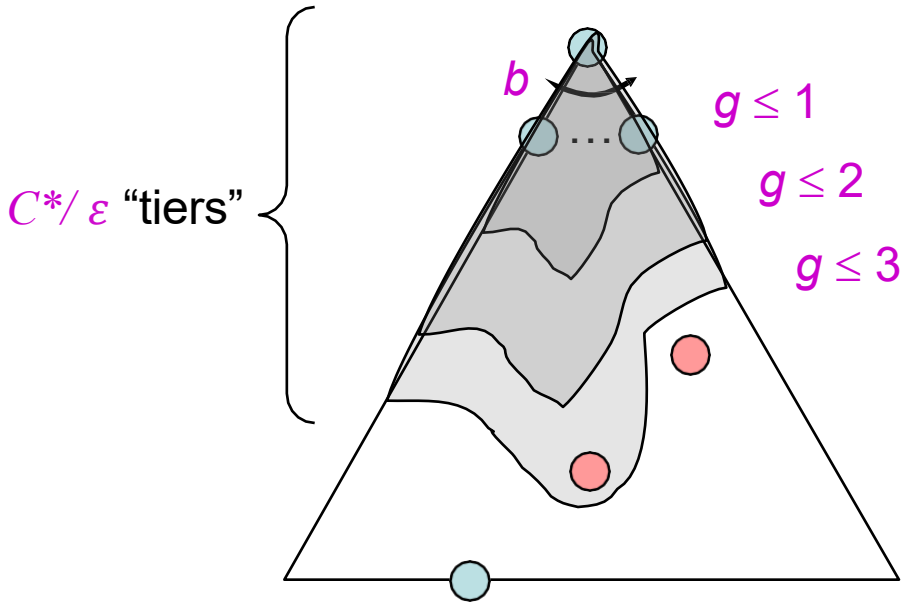
Frontier is a priority queue sorted by g(n)





Uniform Cost Search (UCS) Properties

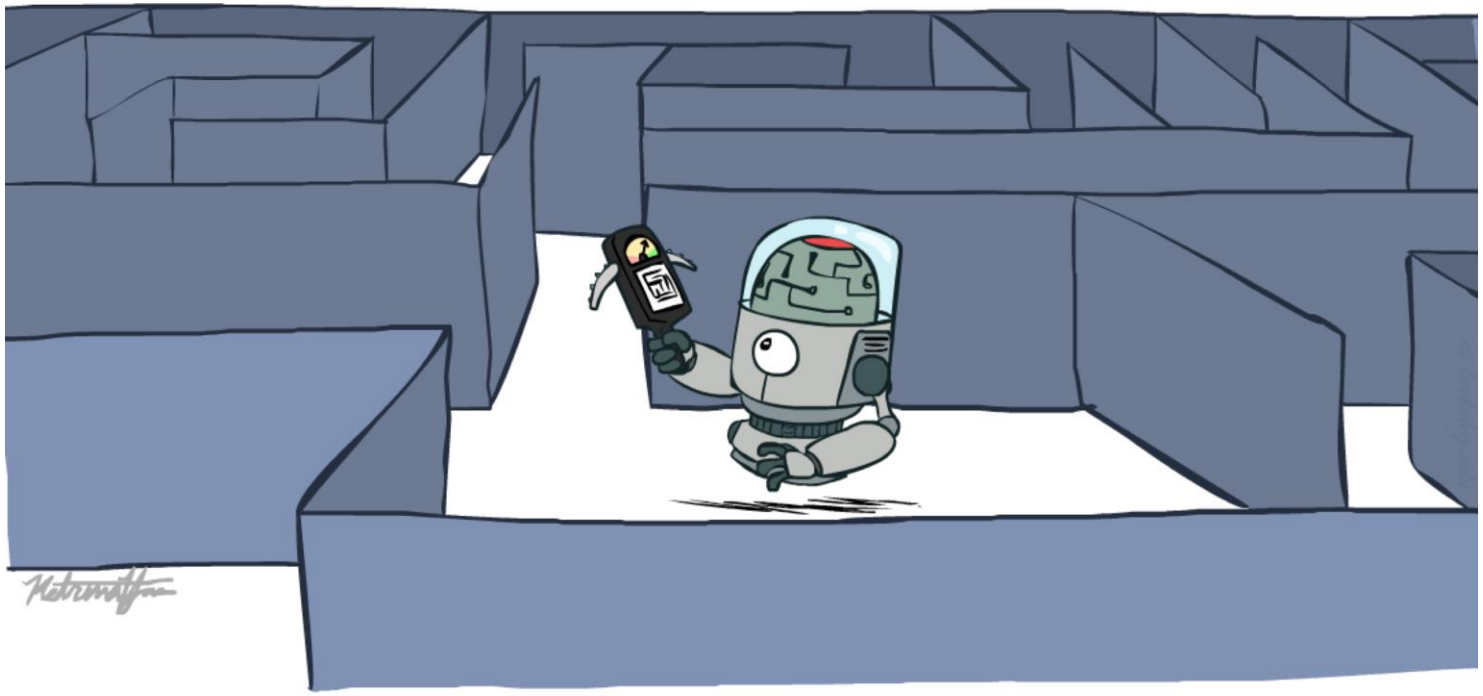
- What nodes does UCS expand?
  - Expands all nodes with cost less than cheapest solution!
  - If the cheapest solution costs C^* and action cost at least ε, then the "effective depth" is roughly C^*/ε
  - Takes time $O(b^{C^*/\varepsilon})$ (exponential in effective depth)
- How much space does the frontier take?
  - Has roughly the last tier, so $O(b^{C^*/\varepsilon})$
- Is it complete?
  - Assuming C^* is finite and $\varepsilon > 0$, yes!
- Is it optimal?
  - Yes!



Summary

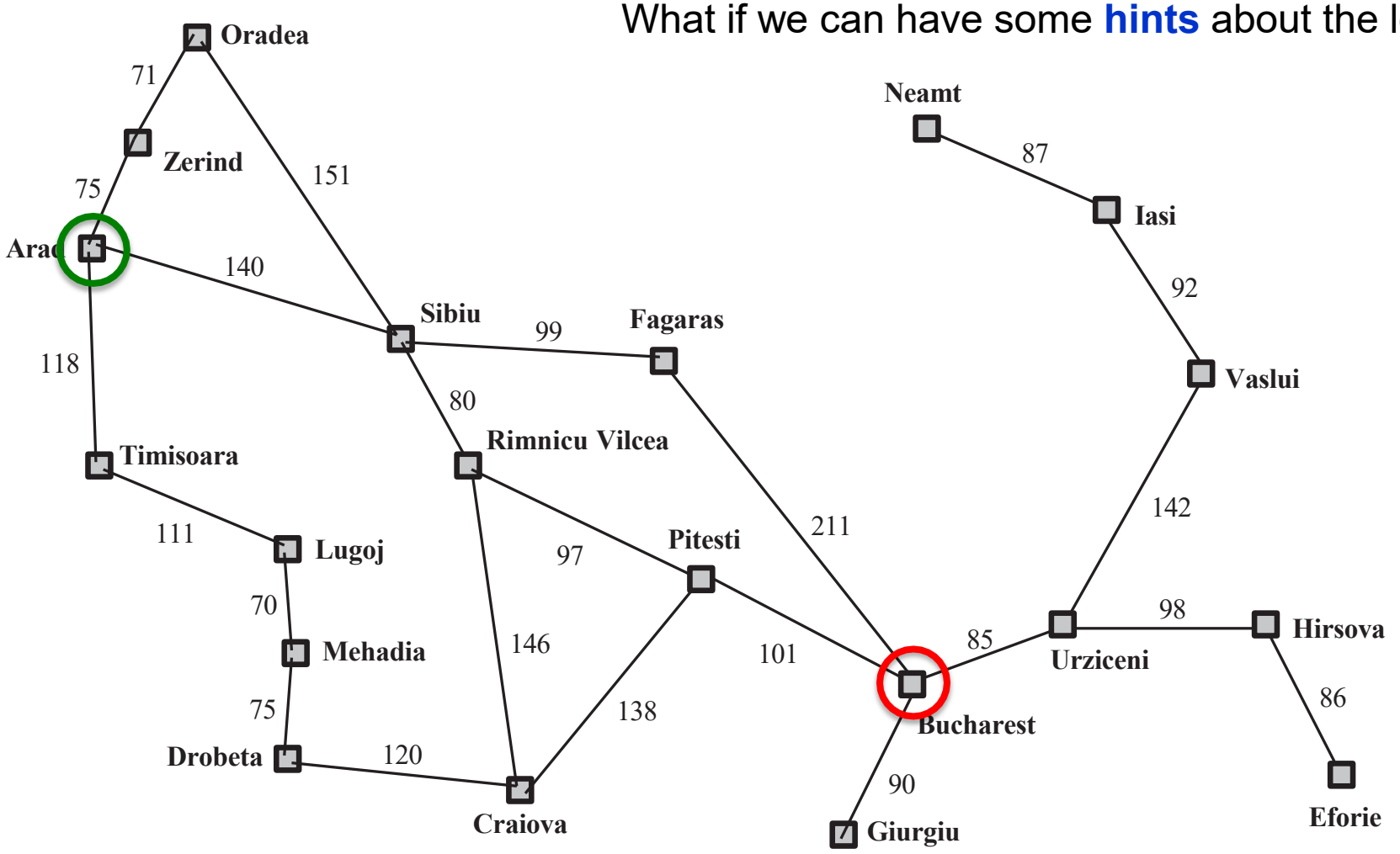
- Assume known, discrete, observable, deterministic, atomic
- Search problems defined by $\mathcal{S}$, s_0, $\mathcal{A}(s)$, $Result(s,a)$, $G(s)$, $c(s,a,s')$
- Search algorithms find **action sequences** that reach goal states
  - Optimal => minimum-cost
- Search algorithm properties:
  - **Depth-first**: incomplete, suboptimal, space-efficient
  - **Breadth-first**: complete, (sub)optimal, space-prohibitive
  - **Iterative deepening**: complete, (sub)optimal, space-efficient
  - **Uniform-cost**: complete, optimal, space-prohibitive

Informed (Heuristic) Search



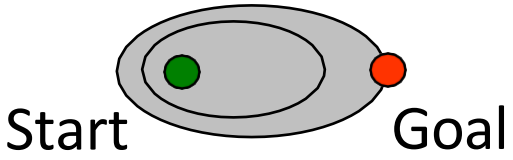
Example: route-finding in Romania

What if we can have some **hints** about the location of goals?

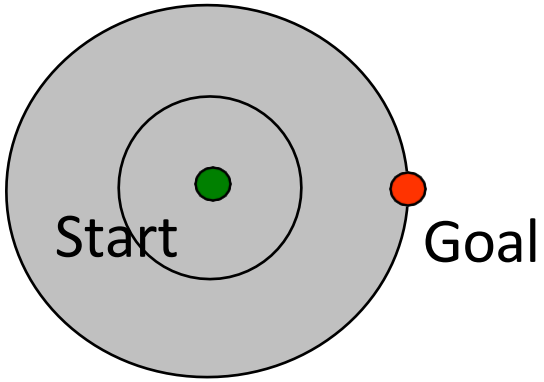


What we would like to have happen

Guide search ***towards the goal*** instead of ***all over the place***



Informed



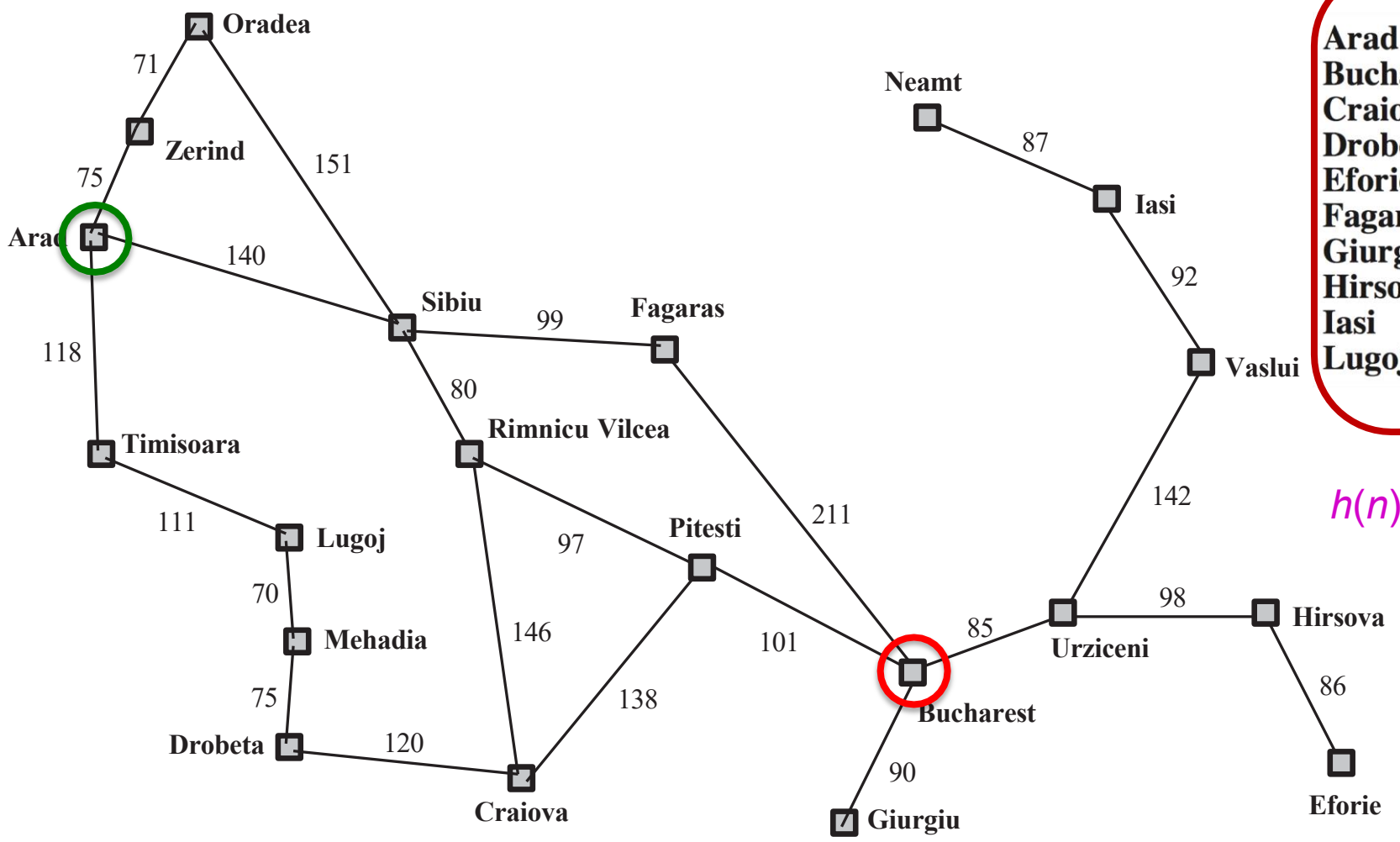
Uninformed

A* Search

A^*: the core idea

- Expand a node n **most likely** to be on an optimal path
- Expand a node n s.t. the cost of the best solution through n is optimal
- Expand a node n with **lowest value** of $g(n) + h^*(n)$
  - $g(n)$ is the cost from root to n
  - $h^*(n)$ is the optimal cost from n to the closest goal
- We **seldom know** $h^*(n)$ but might have a **heuristic** approximation $h(n)$
- A^* = tree search with priority queue ordered by $f(n) = g(n) + h(n)$
  - The most common informed search algorithm

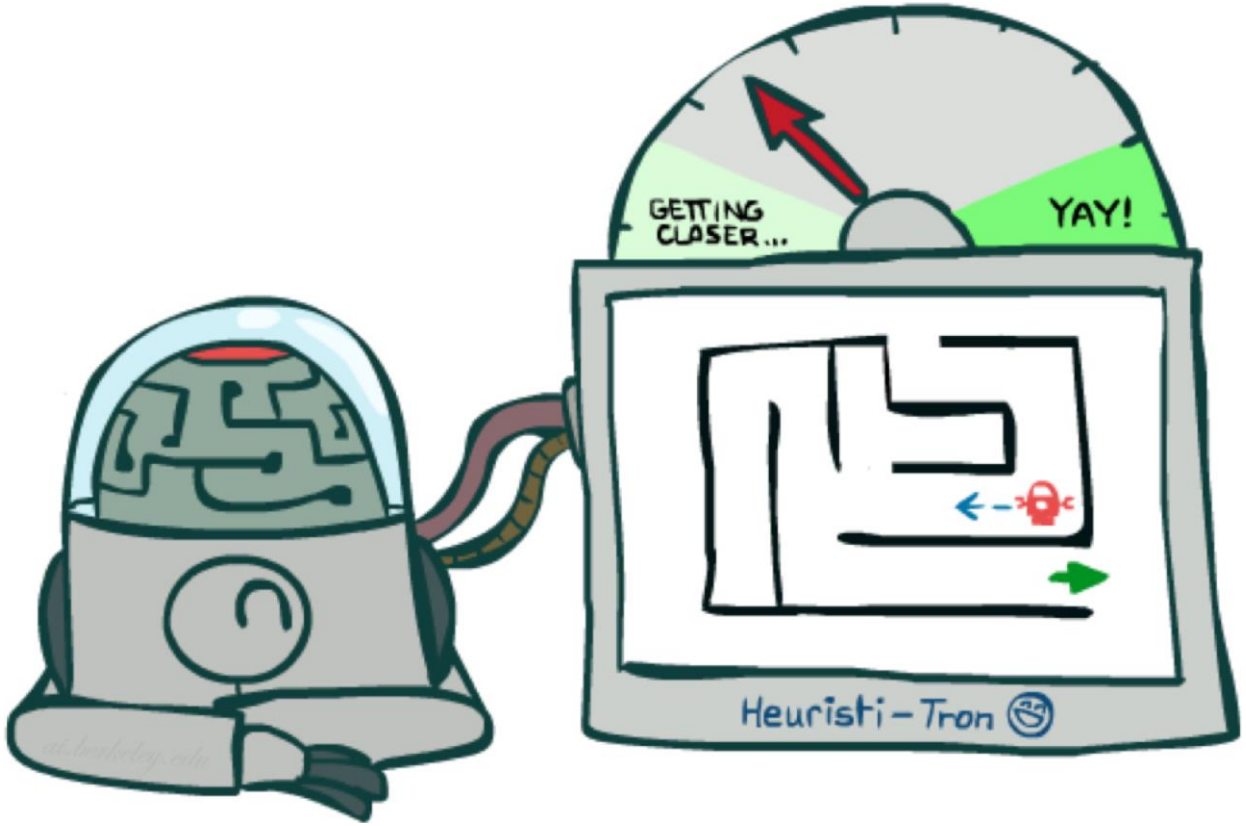
Example: route-finding in Romania



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

$h(n)$ = straight-line distance to Bucharest

Admissible Heuristics



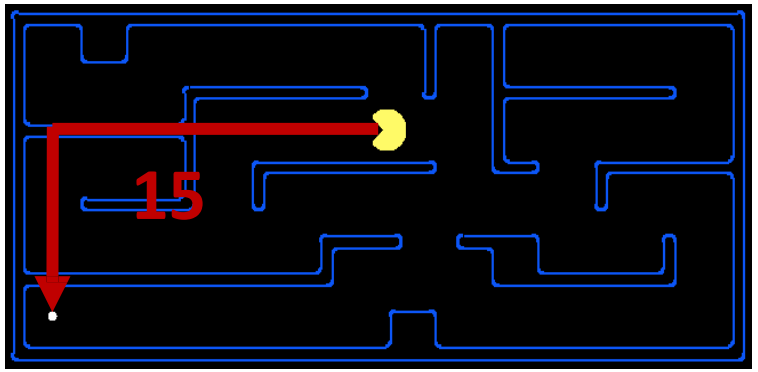
Admissible Heuristics

- A heuristic h is *admissible* (optimistic) if:

$$0 \leq h(n) \leq h^*(n)$$

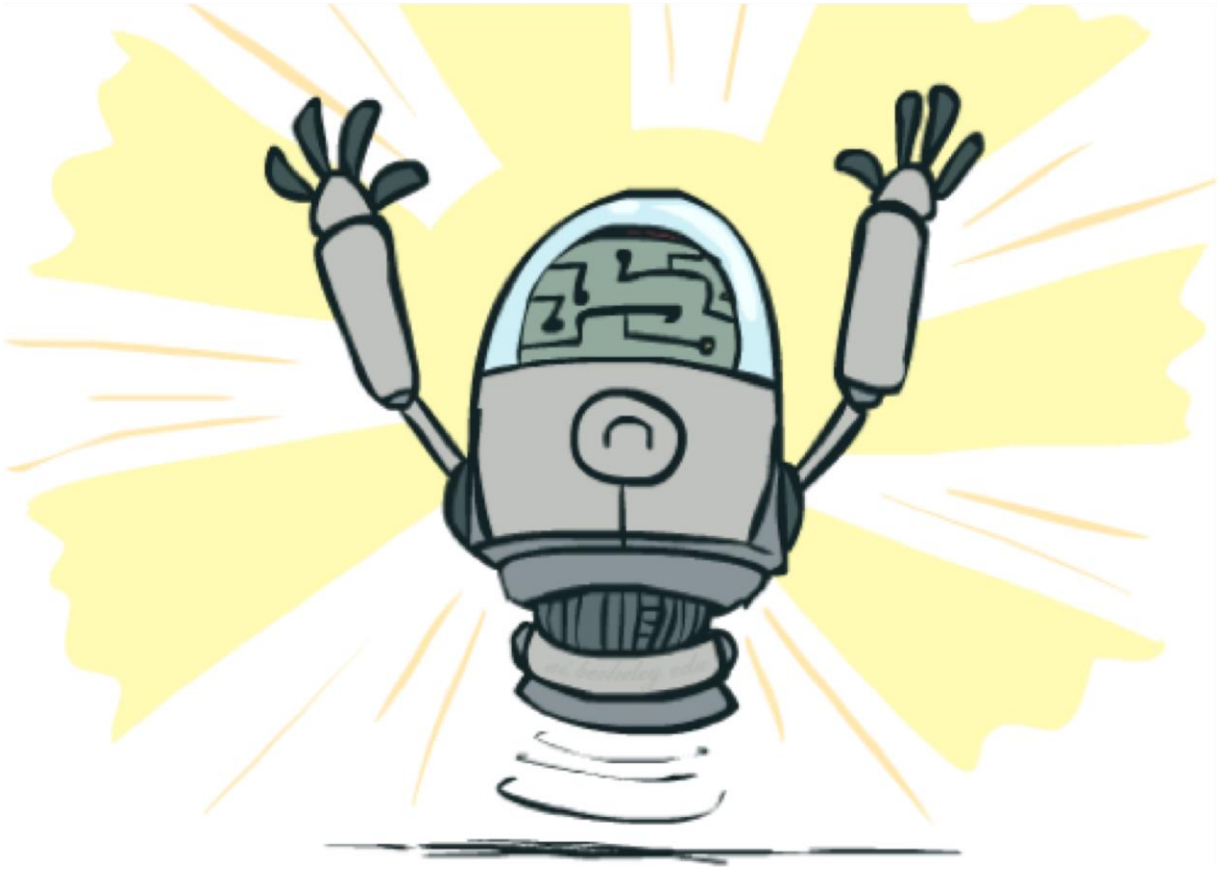
where $h^*(n)$ is the true cost to a nearest goal

- Example:



- Finding **good, cheap admissible** heuristics is the key to success

Optimality of A* Tree Search



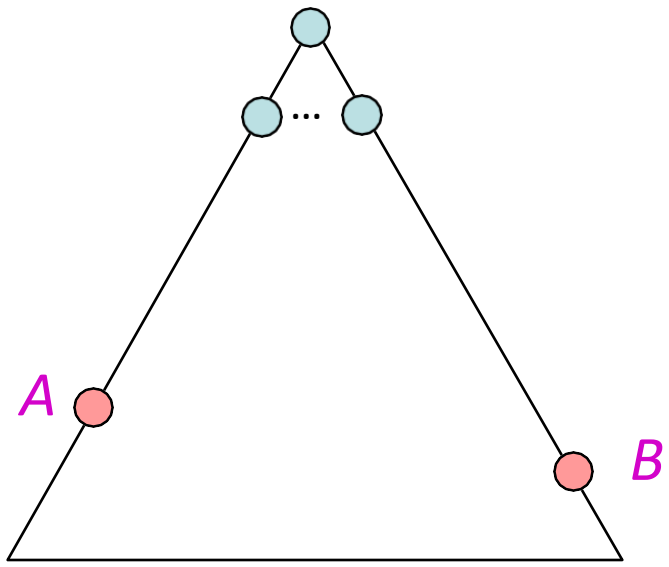
Optimality of A* Tree Search

Assume:

- *A* is an optimal goal node
- *B* is a suboptimal goal node
- *h* is admissible

Claim:

- *A* will be chosen for expansion before *B*

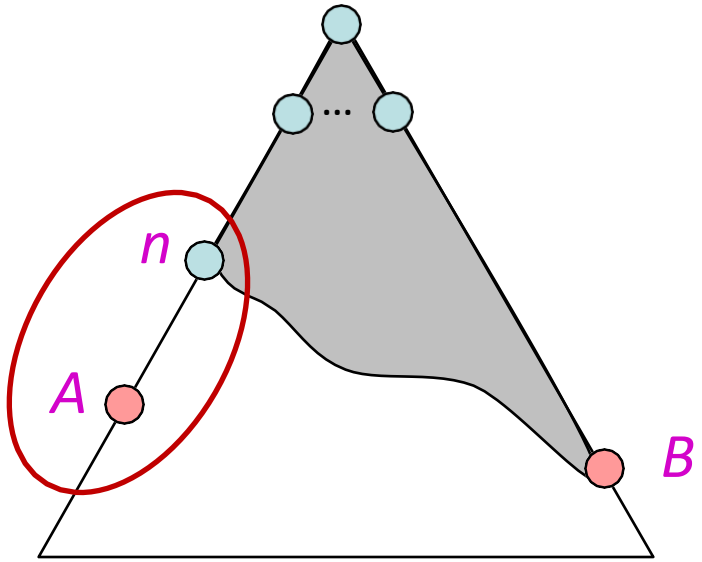


Optimality of A* Tree Search: Blocking

Proof:

- Imagine *B* is on the frontier
- Some ancestor *n* of *A* is on the frontier, too (maybe *A* itself!)
- Claim: *n* will be expanded before *B*

1. $f(n) \leq f(A)$



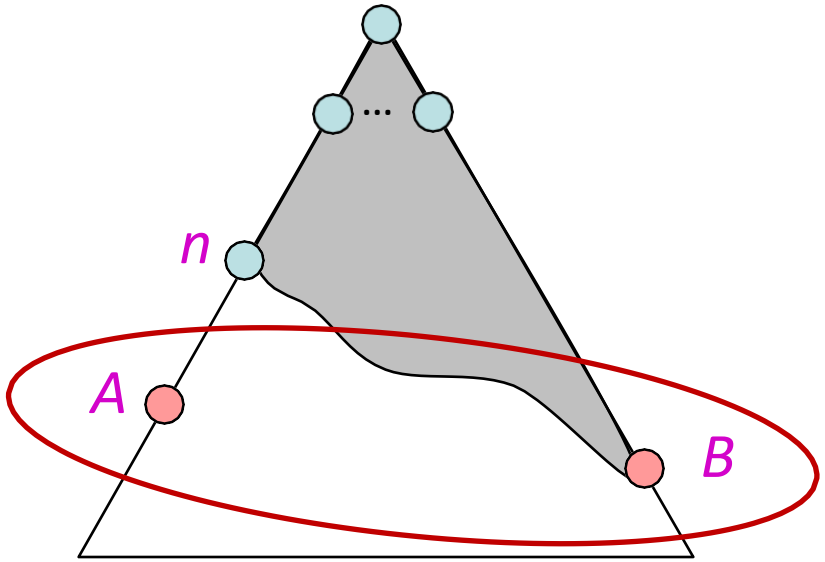
$f(n) = g(n) + h(n)$	Definition of *f*-cost
$f(n) \leq g(A)$	Admissibility of *h*
$g(A) = f(A)$	$h = 0$ at a goal

Optimality of A* Tree Search: Blocking

Proof:

- Imagine *B* is on the frontier
- Some ancestor *n* of *A* is on the frontier, too (maybe *A* itself!)
- Claim: *n* will be expanded before *B*

1. $f(n) \leq f(A)$
2. $f(A) < f(B)$



$g(A) < g(B)$

$f(A) < f(B)$

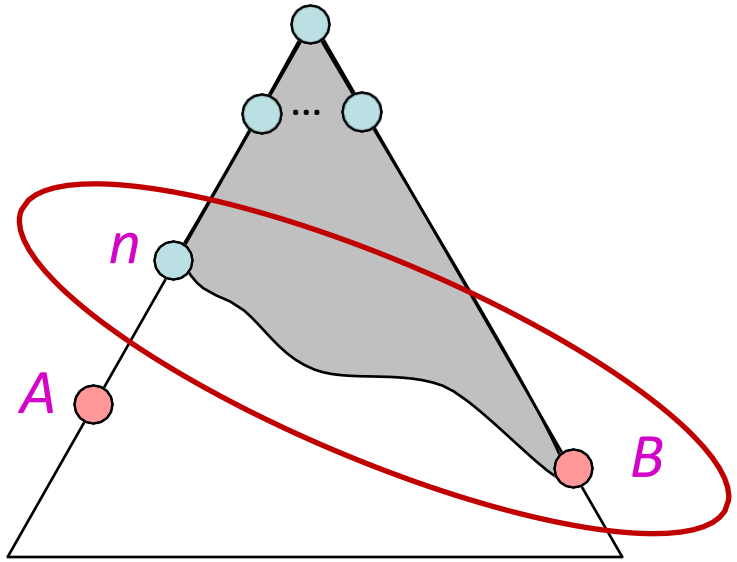
Suboptimality of *B*

$h = 0$ at a goal

Optimality of A* Tree Search: Blocking

Proof:

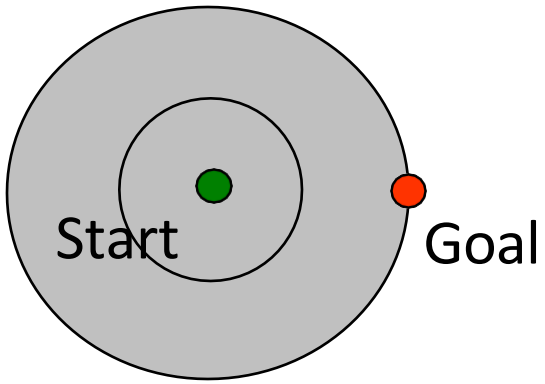
- Imagine *B* is on the frontier
- Some ancestor *n* of *A* is on the frontier, too (maybe *A* itself!)
- Claim: *n* will be expanded before *B*
  1. $f(n) \leq f(A)$
  2. $f(A) < f(B)$
  3. *n* is expanded before *B*
- All ancestors of *A* are expanded before *B*
- *A* is expanded before *B*
- **A* tree search is optimal**



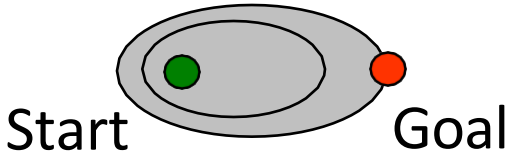
$$f(n) \leq f(A) < f(B)$$

UCS vs A* Contours

- **Uniform-cost** expands equally in all "directions"



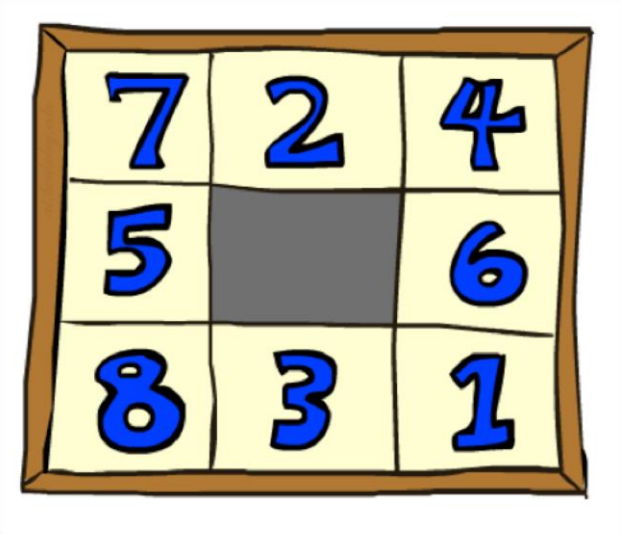
- **A*** expands mainly toward the goal, but does hedge its bets to ensure optimality



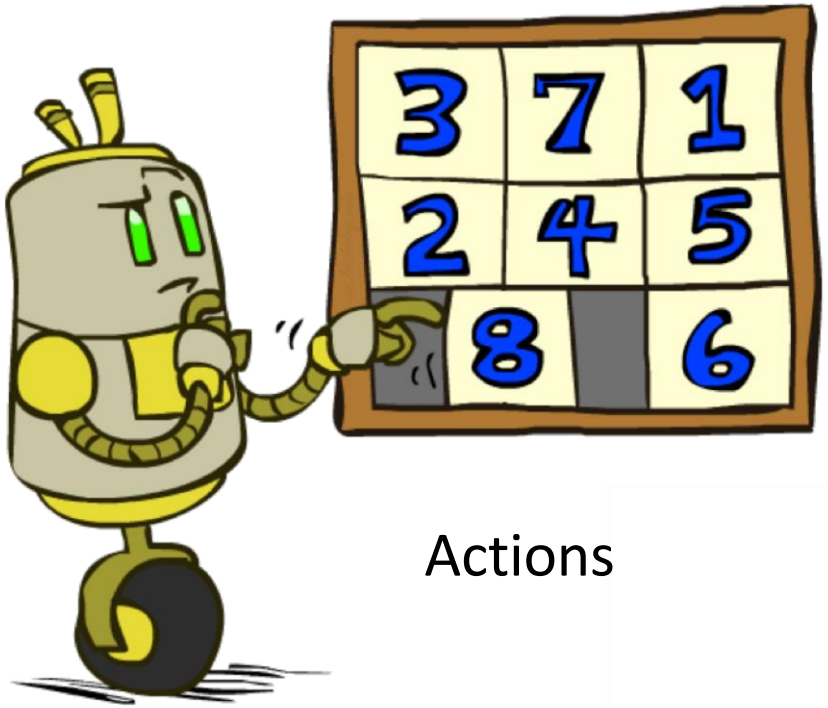
A* Applications

- Video games
- Pathing / routing problems
- Resource planning problems
- Robot motion planning
- Language analysis
- Machine translation
- Speech recognition
- Protein design
- Chemical synthesis
- …

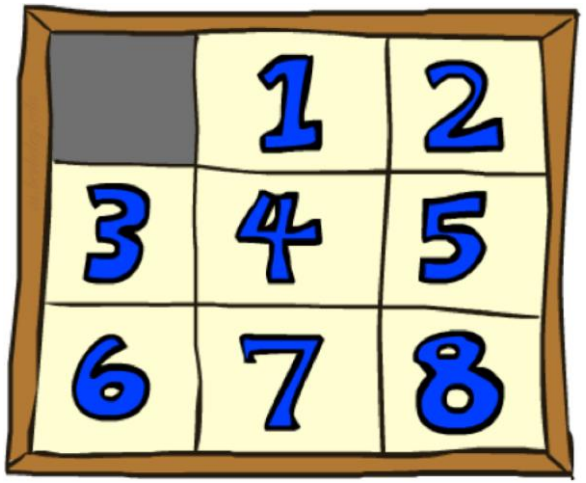
Example: 8 Puzzle



Start State



Actions

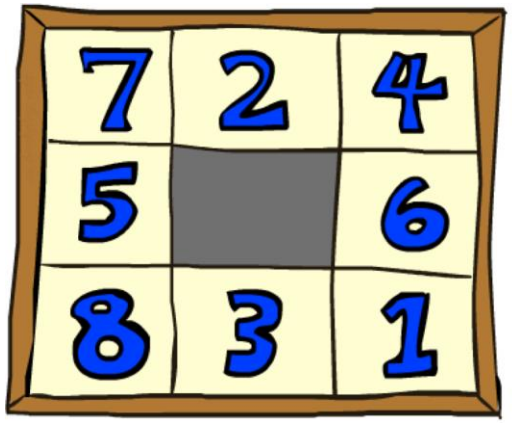


Goal State

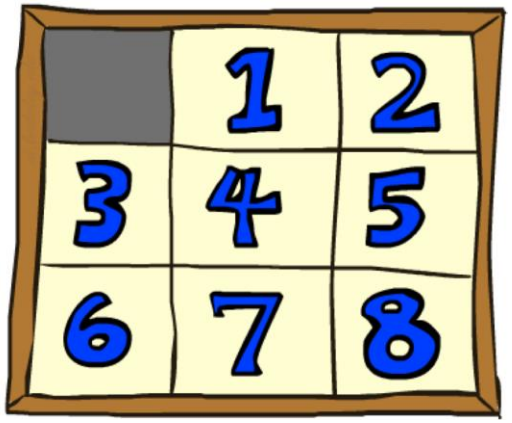
- What are the states?
- How many states?
- What are the actions?
- What are the step costs?

8 Puzzle I

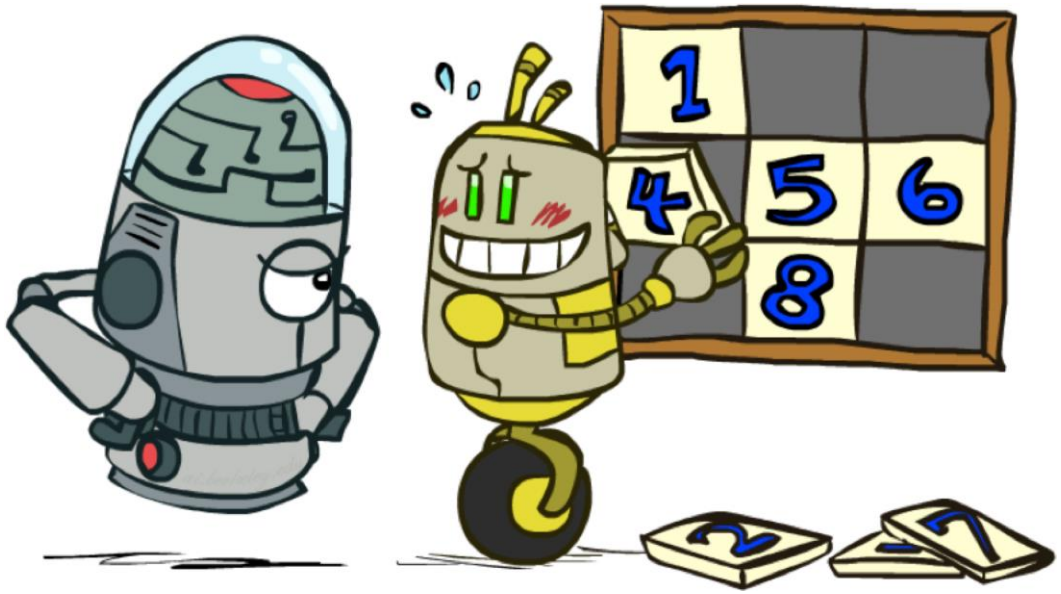
- Heuristic: Number of tiles misplaced
- Why is it admissible?
- h(start) = 8



Start State



Goal State



	Average nodes expanded when the optimal path has…		
	…4 steps	…8 steps	…12 steps
UCS	112	6,300	3.6×10^6
A*TILES	13	39	227

Statistics from Andrew Moore

Combining heuristics

- Dominance: $h_1 \geq h_2$ if

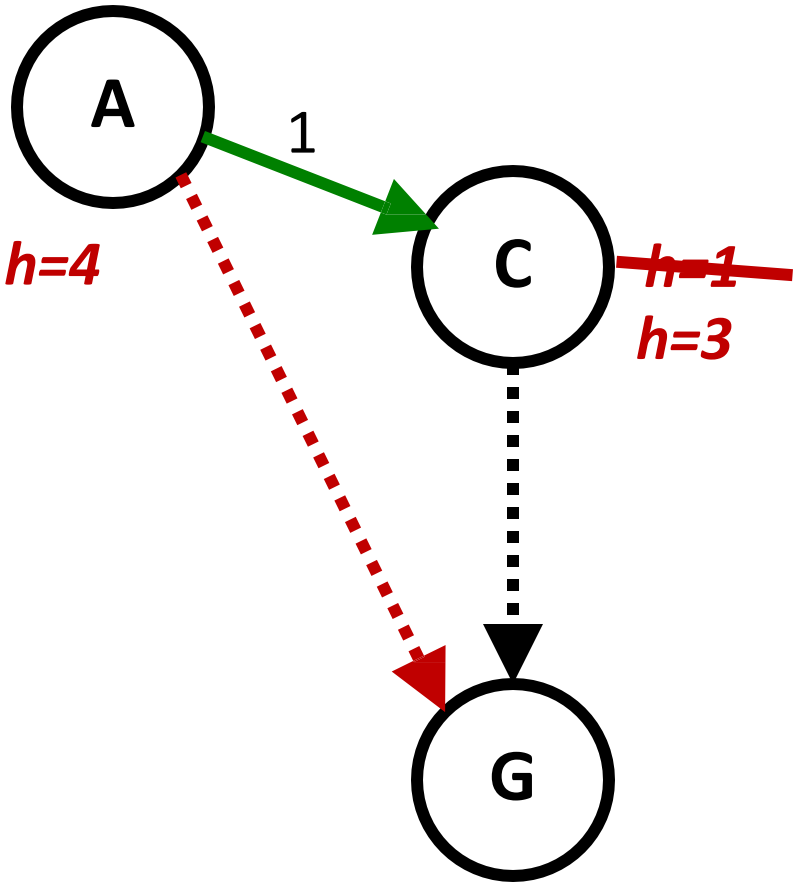
$$\forall n \; h_{1(n)} \geq h_2(n)$$

  - Roughly speaking, larger is better as long as both are admissible
  - The zero heuristic is pretty bad (what does A* do with h=0?)
  - The exact heuristic is pretty good, but usually too expensive!
- What if we have two heuristics, neither dominates the other?
  - Form a new heuristic by taking the max of both:

$$h(n) \; = \; \max(\, h_1(n), h_2(n))$$

  - Max of admissible heuristics is admissible and dominates both!

Consistency of Heuristics



- Main idea: estimated heuristic costs ≤ actual costs
  - Admissibility: heuristic cost ≤ actual cost to goal

    $h(A) \leq h^*(A)$
  - Consistency: heuristic "arc" cost ≤ actual cost for each arc

    $h(A) - h(C) \leq c(A,C)$

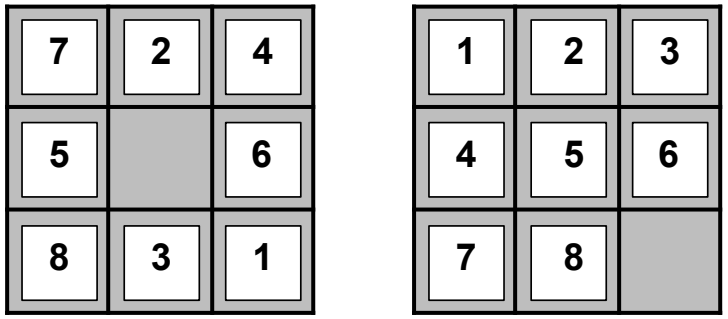
    or $h(A) \leq c(A,C) + h(C)$ (triangle inequality)
    - Note: h* *necessarily* satisfies triangle inequality
- Consequences of consistency:
  - The *f* value along a path never decreases:

    $h(A) \leq c(A,C) + h(C) \quad \Rightarrow g(A) + h(A) \leq g(A) + c(A,C) + h(C)$
- Every consistent heuristic is admissible
  - With a consistent heuristic, A* is cost-optimal

Admissible Heuristics – 8 Puzzle

E.g., for the 8-puzzle:

$h_1(n)$ = number of misplaced tiles
$h_2(n)$ = total Manhattan distance
(i.e., no. of squares from desired location of each tile)



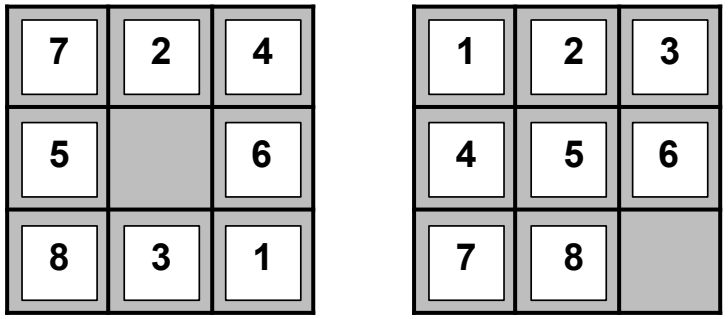
Start State Goal State

$h_1(S)$ =??
$h_2(S)$ =??

Admissible Heuristics – 8 Puzzle

E.g., for the 8-puzzle:

$h_1(n)$ = number of misplaced tiles
$h_2(n)$ = total Manhattan distance
(i.e., no. of squares from desired location of each tile)



Start State Goal State

$h_1(S)$ =?? 6
$h_2(S)$ =?? 4+0+3+3+1+0+2+1 = 14

Dominance – 8 Puzzle

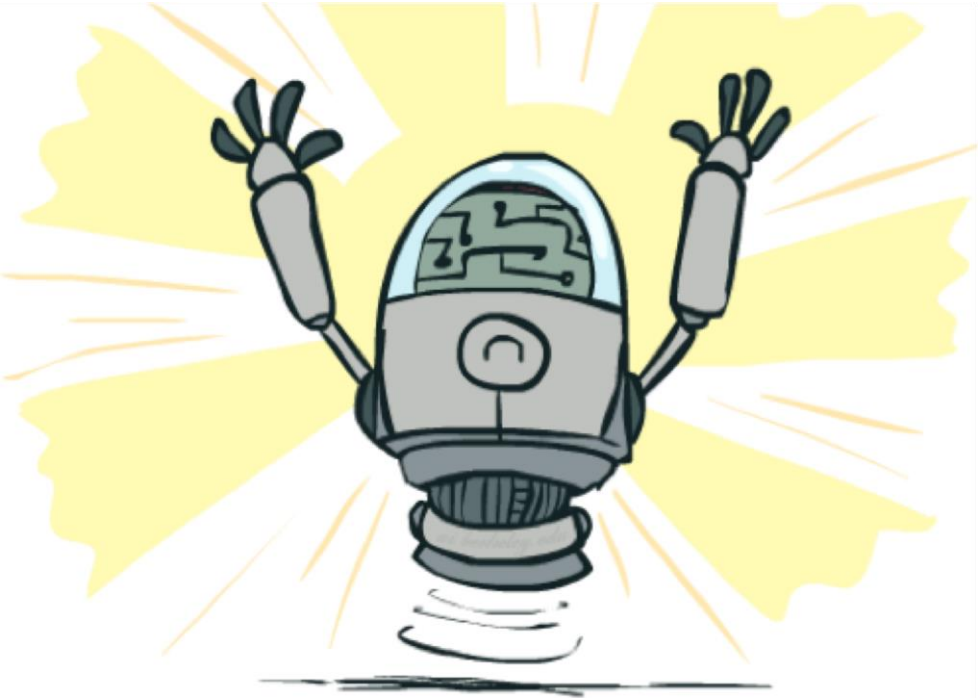
If $h_2(n) \geq h_1(n)$ for all n (both admissible),
then h_2 dominates h_1 and is better for search

Typical search costs:

$d = 14$ IDS = 3,473,941 nodes
A*(h_1) = 539 nodes
A*(h_2) = 113 nodes
$d = 24$ IDS $\approx$ 54,000,000,000 nodes
A*(h_1) = 39,135 nodes
A*(h_2) = 1,641 nodes

Optimality

- A* is optimal if heuristic is admissible
- A* optimal if heuristic is consistent
- Consistency implies admissibility
- Most natural admissible heuristics tend to be consistent, especially if from relaxed problems



But…

- A* keeps the entire explored region in memory
  - => will run out of space before you get bored waiting for the answer
- There are variants that use less memory:
  - IDA* works like iterative deepening, except it uses an *f*-limit instead of a depth limit
    - On each iteration, remember the smallest *f*-value that exceeds the current limit, use as new limit
    - Very inefficient when *f* is real-valued and each node has a unique value
  - RBFS is a recursive depth-first search that uses an *f*-limit = the *f*-value of the best alternative path available from any ancestor of the current node
    - When the limit is exceeded, the recursion unwinds but remembers the best reachable *f*-value on that branch
  - SMA* uses *all available memory* for the queue, minimizing thrashing
    - When full, drop worst node on the queue but remember its value in the parent

A*: Summary

- A* orders nodes in the queue by $f(n) = g(n) + h(n)$
- A* is optimal for trees/graphs with admissible/consistent heuristics

- Heuristic design is key: often use relaxed problems

Acknowledgement

The lecture slides are based on the materials from ai.Berkey.edu

Questions?

UNIVERSITY
OF WOLLONGONG
AUSTRALIA